

基础编译系统用户手册

v0.53

2020 年 07 月 01 日

目 录

第一章 概述	1
1.1 术语	1
1.2 系统简介	1
1.3 文档概述	2
第二章 国产众核基础编译器简要说明	3
2.1 安装说明	3
2.2 使用说明	3
2.3 编译示例说明.....	3
2.4 编译选项简要说明.....	4
2.5 编译器预定义宏和变量.....	5
2.6 数据类型扩展.....	5
2.7 编译器对界支持.....	7
2.8 编译器版本.....	8
第三章 众核存储层次关键字和接口	9
3.1 从核数据 CACHE 相关说明	9
3.2 扩展关键字.....	10
3.3 基础库接口.....	12
第四章 加速线程库	15
4.1 主核加速线程库.....	15
4.2 从核加速线程库.....	25
4.3 主核 IO 空间操作相关接口.....	36
4.4 浮点控制状态寄存器 FPCR 配置接口	38
4.5 加速线程库示例.....	41
第五章 SIMD 向量编程简介	46
5.1 概述	46
5.2 串行程序向量化示例.....	46
5.3 对 C 语言的扩展	48
5.4 程序向量化限制.....	49
第六章 加载器.....	51
6.1 概述	51
6.2 选项介绍	51
第七章 众核编译系统工具.....	55
7.1 混合程序反汇编器.....	55
7.2 LDMREPORT	55
7.3 CHECKLEN	56
7.4 从核栈深度分析功能.....	56
7.5 从核栈在 LDM 上的溢出检查	57

7.6 从核汇编器特殊选项.....	58
7.7 基于控制流图的代码静态特征提取.....	58
7.8 VERSION-SPY 版本检测.....	59
第八章 常用优化支持	60
8.1 循环展开优化.....	60
8.2 乘加融合优化.....	60
8.3 自动向量化.....	60
8.4 结构体重组优化.....	61
8.5 从核 CACHE 预取优化	61
8.6 从核 CACHE 写暗示优化.....	61
8.7 从核 CACHE 属性动态配置.....	61
8.8 从核代码链接时优化.....	63
8.9 从核代码分支预测优化.....	63
8.10 FORTRAN MODULE 访存优化.....	63
8.11 功耗相关优化.....	64
8.12 快速数学库.....	64
8.13 数学库函数细粒度多版本切换.....	64
8.14 支持动态链接程序.....	65
修订记录.....	67

第一章 概述

1.1 术语

GCC: GNU COMPILER COLLECTION, 多语言多平台的开源编译器。

基础编译器: 基于 GCC 源码为神威 E 级高性能计算机 SW39000 众核处理器配备的底层基础编译器, 简称国产众核基础编译器。

加速线程库: 或称 Athread 库, 是针对 SW39000 众核处理器主从加速编程模型所设计的程序加速库, 其目的是为了用户能够方便、快捷地对核组内的线程进行控制和调度, 从而更好地发挥核组内多从核执行的加速性能。

主核: SW39000 众核处理器的运算控制核心 (简称主核)。

从核: SW39000 众核处理器的运算核心 (简称从核)。

从核阵列: SW39000 众核处理器的运算核心阵列 (简称从核阵列)。

核组: SW39000 众核处理器由多个核组构成, 每个核组由一个主核加一个从核阵列组成。

主存: SW39000 众核处理器的内存空间。

LDM: SW39000 众核处理器中从核的快速局部存储空间 (LDM, Local Data Memory)。

DMA: SW39000 众核处理器的主存和 LDM 之间的数据传输的快速通道。

RMA: SW39000 众核处理器核组内的从核 LDM 之间进行的远程数据传输操作, 可以理解为 LDM 空间上核组内从核之间的数据交换。

加速线程: 运行在从核上的程序实体, 简称为线程。

主核代码: 运行在主核上的程序代码, 简称主核程序或主核代码。

从核代码: 运行在从核上的程序代码, 简称从核程序或从核代码。

加载器: 加载器提供对用户众核程序进行加载运行的功能。

1.2 系统简介

基础编译系统是软件系统的重要组成部分, 它是连接 SW39000 众核处理器与应用程序的桥梁。它支持国际标准的编程语言 C、C++ 和 Fortran, 通过基础编译器的编译, 通用的程序可以直接在 SW39000 众核处理器上运行, 并且它也是神威 E 级高性能计算机所有上层语言、并行程序设计环境开发的基础。基础编译系统包括 SW39000 众核处理器基础编译器、基础函数库及二进制工具链等组成部分, 它支持异构众核的 SIMD 扩展编程接口, 提供循环优化、全局优化、过程间分析与优化等传统编译优化支持, 并针对 SW39000 众核处理器的体系结构特点, 进行高效能的编译优化, 从而充分发挥处理器效率。

本系统除提供 SW39000 众核处理器上的编译器支撑外, 还支持用户使用 Athread 程序设计模式。Athread 程序设计主要使用基础编译器提供的基础语言文本和加速线程库进行。加速线程库是针对 SW39000 众核处理器主从加速编程模型所设计的程序加速库, 其目的是为了用户能够方便、快捷地对核组内的线程进行控制和调度, 从而更好地发挥核组内多从核执行的加速性能。与并行 C、MPI、OpenACC* 的程序设计不同, Athread 程序设计着重关注的视角是 SW39000 众核处理器芯片内的并行和映射执行, 并且与芯片底层的联系更加紧密。除了 Athread 加速线程库外, Athread 程序设计还涉及到众核多级存储层次关键字、向量编程接口、基础编译器及相关工具等内容。Athread 程序设计为高级用户深入优化程序、挖掘 SW39000 众核处理器性能潜力提供了更大的可能性, 但同时也具备相对更大的编程和使用难度。

1.3 文档概述

该用户手册主要介绍国产众核基础编译器相关的配置和使用方法，为上层应用和软件开发提供编译方面的参考，为使用基础语言和加速线程库编程的用户提供帮助。

本文余后章节如下安排：

- 第二章对国产众核基础编译器的安装、使用和编译进行简要说明，并介绍了编译器的预定义宏和变量、扩展数据类型及对界支持情况等基础内容。
- 第三章先介绍从核数据 Cache 相关的特点和使用说明，然后介绍 SW39000 众核处理器存储层次相关的关键字和接口，涉及主存空间和 LDM 空间。
- 第四章介绍主从核加速线程库接口，并在最后附上了使用示例。
- 第五章简要介绍向量编程相关内容，以便让用户了解基本的向量编程方法，详细的接口说明和使用方式可参见《基础编译器 SIMD 用户编程手册》。
- 第六章介绍加载器的相关选项说明。
- 第七章介绍众核编译系统上的辅助工具，可以帮助用户了解程序的一些行为和特征，例如目标码反汇编、代码特征提取、栈空间的分析、栈溢出检查、LDM 空间使用情况等。
- 第八章介绍编译系统常用的优化支持，包括循环展开、乘加融合等传统优化，也包括一系列针对架构开发的编译优化。

考虑到用户手册篇幅的因素，本文档对部分内容并未详细描述，将其整理汇总在编译器提供的其他用户文档中。包括：《基础编译器 SIMD 用户编程手册》、《基础编译器常用编译选项手册》。需要了解向量编程、编译选项等详细信息用户请查阅相关文档。

第二章 国产众核基础编译器简要说明

本章主要介绍使用国产众核基础编译器的快速入门方法。本编译器遵守 Unix 和 Linux 编译器标准协议，所生成目标码遵守 Linux ABI 协议，并可以在 SW39000 众核处理器上运行。有了以上的保证，则编译生成的目标文件就可以链接由其他 Linux 上国产众核基础编译器生成的目标文件。

2.1 安装说明

国产众核基础编译器是针对 C，C++，和 Fortran 的优化编译器套装，并提供运行时支持。国产众核基础编译器包含以下内容：

- 支持 SW39000 众核处理器体系架构的 C 编译器
- 支持 SW39000 众核处理器体系架构的 C++编译器
- 支持 SW39000 众核处理器体系架构的 Fortran 编译器
- 说明文档集
- 库
- 调试器
- 二进制工具集

2.2 使用说明

国产众核基础编译器支持 C、C++、和 Fortran 程序，并且通过选项区别支持主核和从核代码生成：

语言	命令名	主核选项	从核选项	混合链接选项
C	swgcc	-mhost	-mslave	-mhybrid
C++	swg++			
Fortran	swgfortran			

注：目前 C++编译器只支持主核代码。

2.3 编译示例说明

a) 独立的主核程序 master.c

swgcc -mhost master.c -o master.out

b) 独立的从核程序 slave.c

swgcc -mslave slave.c -o slave.out

c) 主核程序 master.c、从核程序 slave.c，混合编译

执行：

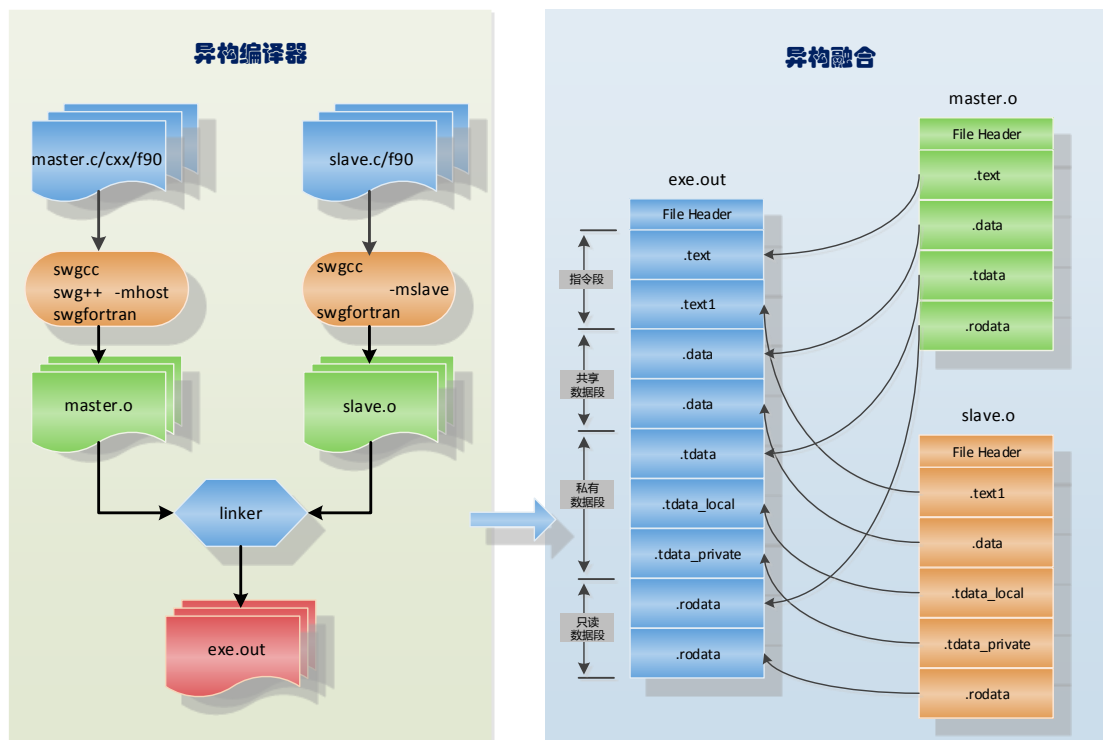
swgcc -mhost -c master.c

swgcc -mslave -c slave.c

swgcc -mhybrid -static master.o slave.o -o exe.out

生成主从混合代码 exe.out。

编译过程如下图所示。



2.4 编译选项简要说明

选项	作用
-S	生成汇编文件
-c	为每个源文件生成一个中间目标文件，但是不进行链接
-g	为之后的调试生成调试符号信息
-I<dir>	为预处理头文件添加查找<路径>
-l<library>	在链接阶段指定需要链接的库文件
-L<dir>	为链接阶段添加查找<路径>
-lm/-lm_slave	在链接阶段使用主从核 libm 数学库，在 C 程序中若使用了 exp(),log(),sin(),cos()函数则需要调用该库
-o<filename>	指定生成的可执行(库)文件的名字
-O3	生成更高级优化的可执行代码
-O0/-O1	生成低于默认优化的可执行代码（默认为-O2）

注：更多的可用选项以及说明请参考《基础编译器常用编译选项手册》，或使用“man gcc”帮助文档。

2.5 编译器预定义宏和变量

2.5.1 预定义宏

swgcc 编译器内置了一些宏，灵活的使用这些预定义宏对用户编写异构程序是有用的：

- 1、以下宏在-mhost 选项下有效：

`__sw_host__`

- 2、以下宏在-mslave 选项下有效：

`__sw_slave__`

- 3、以下宏在-msimd 选项下有效：

`_SIMD_HOST`（-mhost 下）

`_SIMD_SLAVE`（-mslave 下）

- 4、以下宏与选项无关，均有效：

`__TARGET_sw_64__`

`__sw_64`

`__sw_64__`

用户可以使用-dM -E 这两个编译器选项组合来打印所有预定义宏。

2.5.2 从核默认局存变量

使用方法：`#include<slave.h>`

- 1、变量名称：`_PEN`

变量说明：从核逻辑号（0-63）。

- 2、变量名称：`_ROW`

变量说明：从核所在行号（0-7）。

- 3、变量名称：`_COL`

变量说明：从核所在列号（0-7）。

- 4、变量名称：`_CGN`

变量说明：从核所在核组号（0-5）。

- 5、变量名称：`_cache_size`

变量说明：从核当前所配置的 cache 大小（0、32、128）KB。

- 6、变量名称：`_ldm_share_mode`

变量说明：从核连续共享 LDM 模式（0/单簇、1/双簇、2/四簇、4/全阵列簇、5/全阵列行）。

- 7、变量名称：`_ldm_share_size`

变量说明：从核连续共享 LDM 大小（0、4、8、16、32、64、128）KB。

2.6 数据类型扩展

基于 SW39000 众核处理器的体系结构特点以及指令集，在标准的 C、C++和 Fortran 语

法标准之外，基础编译系统扩展了部分数据类型，主要是向量数据类型以及从核的半精度浮点数等。

从核的半精度浮点数据支持两种格式，即 IEEE-754 标准格式和 BF16 格式。IEEE-754 标准格式 16 位数据包含 1 位符号、5 位指数、10 位尾数，浮点数表示范围为 $6.103515625 \times 10^{-5} \sim 65504$ ；BF16 格式 16 位数据包含 1 位符号、8 位指数、7 位尾数，浮点数表示范围为 $1.17549435 \times 10^{-38} \sim 3.38953139 \times 10^{38}$ 。编译器默认使用 IEEE-754 标准格式，编译及链接时加 -mslave-bf16 选项则使用 BF16 格式，在同一程序中建议只使用一种格式，否则会产生不可预知的问题。

	数据类型	关键字	备注
主核	定点整形向量	intv8	8 个 int 类型组成的向量
		uintv8	8 个 unsigned int 类型组成的向量
	定点长整形向量	int256	256 位长整形向量
		uint256	无符号 256 位长整形向量
	单精度浮点向量	floatv4	4 个单精度浮点类型组成的向量
		v4real(kind=4) (Fortran)	
	双精度浮点向量	doublev4	4 个双精度浮点类型组成的向量
		v4real(kind=8) (Fortran)	
从核	半精度浮点标量	_Float16/float16	若使用 float16，从核程序中需要 #include<slave.h> 默认使用 IEEE-754 标准格式，编译及链接时加 -mslave-bf16 选项则使用 BF16 格式。 打印半精度浮点数据需强制转换成 float 类型。 支持半精度浮点标量的加(+)、减(-)、乘(*)、除(/)、平方根(sqrth)运算。
	定点整形向量	intv16	16 个 int 类型组成的向量
		uintv16	16 个 unsigned int 类型组成的向量
	定点长整形向量	int512	512 位长整形向量
		uint512	无符号 512 位长整形向量
	单精度浮点向量	floatv8	8 个单精度浮点类型组成的向量
		v8real(kind=4) (Fortran)	
	双精度浮点向量	doublev8	8 个双精度浮点类型组成的向量
		v8real(kind=8) (Fortran)	
	半精度浮点向量	float16v32	32 个 _Float16 类型组成的向量 支持半精度浮点向量的加(+)、减(-)、乘(*)、除(/)、平方根(simd_vsqrth)运算。

2.7 编译器对界支持

2.7.1 支持的默认对界

1、使用-faddress_align=*n* 编译，则向量变量、数组、结构体、联合体等的首地址默认对界为 *n* 字节，其他普通全局变量按照定义变量类型对界；

2、不使用-faddress_align=*n* 编译，则按照定义变量类型对界；

3、主核函数初始栈指针保持 32 字节对界；

4、从核函数初始栈指针保持 64 字节对界。

5、2.7.2 节提供的用户自定义属性设置功能，其优先级高于本节的-faddress_align=*n* 选项。

变量类型	对界（字节）
向量变量、数组、结构体、联合体首地址（使用-faddress_align= <i>n</i> 编译）	<i>n</i>
标量全局变量（使用-faddress_align= <i>n</i> 编译）	按照定义类型对界
标量变量、数组、结构体、联合体首地址（不使用-faddress_align= <i>n</i> 编译）	按照定义类型对界
函数初始栈指针	主核 32，从核 64

2.7.2 用户自定义的对界

2.7.2.1 C 语言对界属性设置

`__attribute__((aligned(align_size)))`

注意：align_size 为 2 的整数次幂，使用如上 attribute 可用于修饰全局变量或者函数内局部变量。

例如：

```
long tt[100] __attribute__((aligned(64)));
int fun(){
    struct xx xxt __attribute__((aligned(512)));
    .....
}
```

2.7.2.2 FORTRAN 语言对界属性设置

目前支持对数组类型进行用户自定义对接属性设置，方法如下：

`real(kind=8),dimension(16),align(align_size)::aa`

或者

`real(kind=8),align(align_size)::aa(16)`

在数组声明时，增加关键字 align，align_size 应为 2 的整数次幂。需要注意的是，对于

如下的使用方式:

```
real(kind=8),align(32)::aa(16), b
```

编译器只保证数组 `aa` 的首地址为 32B 对界, 对于普通变量 `b` 的地址不做对界调整。

另一方面, 鉴于很多历史遗留的 FORTRAN 课题中使用了 `COMMON` 块来共享内存空间, 编译器对 `COMMON` 块中包含设置了对界属性的数组的这种情况的对界作如下设置:

```
integer, align(128) :: aa(7), bb, cc(22), dd
```

```
real, align(256) :: ee(13), ff, gg(24), hh
```

```
COMMON / COMM_main1 / aa, cc, dd, ee, hh
```

```
COMMON / COMM_main2 / bb, ff, gg
```

因为 `COMMON` 的本意就是其 `common` 块中的所有元素紧密排列, 因此编译器将 `COMMON` 块的起始地址按照其中所有元素的最大对界值进行对界设置, 而 `COMMON` 块中的各个元素按照起始地址紧密排列。

在上面的例子中, 数组 `aa, cc` 要求 128B 对界, 数组 `ee, gg` 要求 256B 对界, `COMM_main1` 中最大的对界值是数组 `ee` 的 256B 对界, 因此分配的 `COMM_main1` 的起始地址是满足 256B 对界的, 而因为 `aa, cc, dd, ee, hh` 紧密排列, 可以计算出其中的数组 `ee` 的起始地址必定无法满足声明时的 256B 对界。同理 `COMM_main2` 的起始地址满足 256B 对界, 数组 `gg` 的起始地址计算后只有 8B 对界。

2.8 编译器版本

如果编译时用选项 `-v`, 编译器则会给出自身的版本信息, 例如:

```
$ swgcc -v
```

```
gcc version 7.1.0 20170502 (swgcc-419 Committer: wuw <ww7tc163@163.com> by comp  
at localhost.localdomain on Thu May 16 10:56:51 CST 2019) (GCC)
```

第三章 众核存储层次关键字和接口

3.1 从核数据 Cache 相关说明

基础编译系统面向的 SW39000 众核处理器提供了从核数据 Cache，存储层次关键字、基础库接口、加速线程库接口等方面的设计都与之相关。因此，这里首先对从核 Cache 的相关结构和特征加以说明，用以辅助用户使用和理解。

3.1.1 从核数据 Cache 结构

每个从核具有一块高速的本地局部数据存储空间 LDM，LDM 总容量为 256KB。该空间可配置成完全由软件管理，也可部分容量配置成硬件自动管理的数据 Cache，两种管理方式的容量配比分档可调。配置成软件管理的局部存储时，从核可以使用访问 LDM 空间的 ld/st 的方式加以访问，也可以使用发起异步 DMA 或 RMA 的方式实现 LDM 与外界的数据批量交换；当部分空间配置为一级数据 Cache 时，从核可以通过访问主存可 Cache 空间的 ld/st 的方式对该空间进行 Cache 访问，Cache 行大小为 256 字节。硬件也支持对主存空间的不可 Cache 访问。

硬件支持 0KB、32KB 和 128KB 三档数据 Cache 容量配置。用户可以通过加载器的 -c 选项进行配置，若不设置该选项，加载器默认配置从核数据 Cache 容量为 0KB。

为避免从核组数据 Cache 访问共享主存的假共享问题，硬件支持带 Mask 的写机制，最小粒度为 4B。这种机制可保证：当一个 Cache 行内的数据只有部分内容需要写回时，硬件采用原子读改写的主存访问方式进行处理，保证了假共享时的数据正确性，当然其缺点是导致访存带宽的浪费。

为提升从核 Cache 的好用性和效率，硬件提供了淘汰、刷新、写暗示、配置私有段区间、预取等指令支持。淘汰和刷新用于从核 Cache 的管理，淘汰将指令确定的数据流地址对应的主存数据在本从核数据 Cache 中内容淘汰，刷新对本从核数据 Cache 中指定位置的 Cache 行执行刷新回主存的操作。

写暗示指令提示包含寻址字节的对界 256 字节块的内容将不再有读访问，但不久将有写操作。从核将不读存储器以前的内容而直接分配并占有该 Cache 行，Cache 行的内容可设为任意值。写暗示避免从主存中载入无用的数据，提高了处理性能。

配置私有段区间指令将映射到该私有段区间内的 Cache 行以整 Cache 行方式写回，用于改善写回主存性能；预取指令将主存中对应地址的数据预取到从核 Cache 中，隐藏 Cache 访问延迟。

编译系统针对这些硬件指令提供了相应的功能接口或编译优化的支持。

3.1.2 Cache 一致性

3.1.2.1 单边一致性

从核 Cache 不保证与主核 Cache 的一致性。主核访存过程中无法从从核 Cache 中获得最新数据；但从核淘汰或装填时，能够获取主核 Cache 中的最新数据。

3.1.2.2 垂直一致性

从核淘汰或装填时，能够获取主核 Cache 中的最新数据，但从核 Cache 不保证与其它从

核 Cache 的一致性。从核访存过程中，始终无法从其他从核的 Cache 中获得最新数据。

3.1.2.3 Cache 和 LDM

从核 Cache 不保证与软件管理的 LDM 空间的一致性。从核 DMA 过程中，无法获得从核数据 Cache 中的最新数据。使用 DMA 时需要用户自行保证一致性，使用不可 Cache 空间或使用刷新淘汰 Cache 等操作。

3.1.2.4 屏蔽码机制

为避免由于从核 Cache 不保证一致性带来的假共享问题，从核 Cache 对共享主存空间的写回支持带屏蔽的细粒度写机制，屏蔽粒度为 4B，一个 Cache 行中只有确实被修改过的数据才会写入主存（读改写方式处理），如果整个 Cache 行都被修改，则不使用读改写方式。

为了提升主存写回的效率，避免浪费带宽，在以下情况下硬件不使用带 Mask 的写机制：

1) 对于主存虚空间中属性为“从核私有”的写回操作，硬件采用整 Cache 行写回主存的方式；

2) 对于主存虚空间中属性为“从核共享”的写回操作，软件可以将其中部分空间配置成私有段区间，对该区间内的可 Cache 访问，硬件采用整 Cache 行写回主存的方式；

上面两点的结果对软件意味着：从核私有空间和共享空间的私有段可以以任意数据粒度访问；而对共享空间，软件必须保证小于 4B 粒度数据访问的正确性。

3.1.2.5 硬件解决假共享

主主之间、主从之间、从从之间，硬件均保证对共享主存的同一个 Cache 行的假共享的数据一致性（访问粒度 4B 及以上）。

3.1.2.6 软件保证一致性的操作

对于“从核 Cache-从核 Cache”、“从核 Cache-DMA 操作访问”或“从核 Cache-主核 Cache”之间存在真共享数据且最新副本在从核/主核 Cache 中，或“从核 Cache-从核 Cache”之间存在独占 Cache 行所有权限转移时，软件必须显式进行如下操作序列：

- 1) 对数据所有者的从核/主核的 Cache 数据进行刷新；
- 2) 通过 memb 保证刷新完成；
- 3) 需要一致性的双方通过硬件同步或主存空间上的原子操作进行同步。

3.2 扩展关键字

3.2.1 针对主存的空间编制和关键字

主存虚空间	空间属性	可 cache	空间编址	使用实例
私有空间 (读写)	从核私有	是	0x4000,0000,0000	从核私有变量 C语言： <code>__thread int aa[100];</code>
				FORTTRAN： <code>common /private_*/ aa</code>

				!\$omp threadprivate (private_*/)
阵列共享 (只读)	代码段和rodata		0x4fff,f000,0000	从核指令
阵列共享 (读写)	数据段		0x5000,0000,0000	默认全局变量 int aa[100];
交叉段 (读写)	段名: .cross_rw (初始化变量) .cross_un(未 初始化变量)		0x5800,0000,0000	C语言: __cross int aa;
				Fortran: common /cross_*/ aa !\$omp threadprivate (cross_*/)
私有空间 (读写)	从核私有 (不可Cache空间)		0x6000,0000,0000	从核私有不可cache变量 C语言: __uncached __thread int aa;
				Fortran: common /uprivate_*/ aa !\$omp threadprivate (uprivate_*/)
阵列共享 (只读)	代码段和rodata	否	0x6fff,f000,0000	
阵列共享 (读写)	数据段		0x7000,0000,0000	不可cache全局变量 C语言: __uncached int aa;
				Fortran: common /uncached_*/ aa !\$omp threadprivate (uncached_*/)
交叉段 (读写)	段名: .cross_rw (初始化变量) .cross_un(未 初始化变量)		0x7800,0000,0000	C语言: __uncached __cross int aa;
		Fortran: common /ucross_*/ aa !\$omp threadprivate (ucross_*/)		

注：使用 **fortran** 对应的关键字，编译时需要选项 **-fopenmp**

3.2.2 针对 LDM 的空间编制和关键字

LDM 虚空间	空间属性	空间编址	对应段名	使用实例
私有 LDM	本从核LDM常驻空间	0x0开始	.ldm_fix	C语言: __thread_local_fix aa;
				Fortran: common /local_*/ aa
	本从核LDM空间	LDM 常驻空间之后	.ldm	C语言: __thread_local aa[100];

				Fortran: <code>common /local_*/ aa</code>
	本从核LDM空间, 多个kernel间共享	LDM 常驻空间之后	<code>.ldm_reuse_*</code>	C语言: 不帶共享段名的方式 (都共享同一片LDM空间): <code>__thread_kernel(kname) int aa[100];</code> 或帶共享段名的方式 (同一个共享段名修饰的共享同一片空间): <code>__thread_kernel(kname,reusename) int xx1, aa1[100];</code> Fortran: 暂不支持
连续共享 LDM	本从核阵列LDM空间, 空间大小由“64*每个从核共享空间容量”限制	0x3000,0000,0000	<code>.ldm_share</code>	C语言: <code>__thread_local_share int aa[100];</code>
				Fortran: <code>common /slocal_*/ aa</code>
分布共享 LDM	本从核阵列 LDM 空间, 其中 VA[17:0]为 LDM 内地址, VA[25:20]为目标从核号, 空间不连续	0x2000,0000,0000		用户自行定义使用

备注: 共享 ldm 空间要求 4 字节对齐访问, 编译器-O3 下会把程序里面连续赋值的操作优化成调用 `memset` 函数, 地址或长度不是 4 字节倍数时 `memset` 函数里会有字节访问, 在这种情况下不希望优化成 `memset`, 这时共享 ldm 数组声明时可以使用 `volatile` 修饰。

3.3 基础库接口

3.3.1 主存分配

主核接口:

接口名称	功能	从核可Cache属性	空间说明
<code>malloc</code>	申请主存连续共享段	是	返回 0x5000,0000,0000 为起始地址的空间
<code>libc_aligned_malloc</code>	申请主存连续共享段	是	返回 0x5000,0000,0000 为起始地址的空间, 返回地址 64 字节对齐
<code>libc_aligned_free</code>	释放主存连续共享段		与 <code>libc_aligned_malloc</code> 配合使用
<code>libc_uncached_malloc</code>	申请主存连续共享段	否	返回 0x7000,0000,0000 为起始地址的空间
<code>libc_uncached_free</code>	释放主存连续共享段		与 <code>libc_uncached_malloc</code> 配合使用
<code>libc_uncached_aligned_malloc</code>	申请主存连续共享段	是	返回 0x7000,0000,0000 为起始地址的空间, 返回地址 64 字节对齐

libc_uncached_aligned_free	释放主存连续共享段		与 libc_uncached_aligned_malloc 配合使用
----------------------------	-----------	--	-------------------------------------

从核接口：

接口名称	功能	从核可Cache属性	空间说明
malloc	申请主存从核私有空间	是	返回0x4000,0000,0000为起始地址的空间
libc_aligned_malloc	申请主存从核私有空间	是	返回0x4000,0000,0000为起始地址的空间，返回地址64字节对齐
libc_aligned_free	释放主存从核私有空间		与libc_aligned_malloc配合使用
libc_uncached_malloc	申请主存从核私有空间	否	返回0x6000,0000,0000为起始地址的空间
libc_uncached_free	释放主存从核私有空间		与 libc_uncached_malloc 配合使用
libc_uncached_aligned_malloc	申请主存从核私有空间	是	返回0x6000,0000,0000为起始地址的空间，返回地址64字节对齐
libc_uncached_aligned_free	释放主存从核私有空间		与 libc_uncached_aligned_malloc 配合使用

3.3.2 从核局存分配

从核 LDM 支持精简快速的动态分配，具体接口如下：

3.3.2.1 get_allocatable_size

函数原型：

`unsigned int get_allocatable_size(void)`

函数说明：

得到当前可动态分配的 LDM 空间大小，也即局存动态分配运行时库管理的 LDM 空间大小。

参数说明：

无

返回值：

当前可分配的 LDM 空间大小，单位为字节。

3.3.2.2 ldm_malloc

函数原型：

`void* ldm_malloc(size_t size)`

函数说明：

局存空间申请，支持 Fortran 中调用。

参数说明:

size: 以字节为单位的空间大小。

返回值:

成功返回分配 LDM 空间的起始地址;

失败返回 NULL。

3.3.2.3 ldm_free

函数原型:

```
void ldm_free(void *addr, size_t size)
```

函数说明:

局存空间释放，释放空间起始地址为 addr。支持 Fortran 中调用。

参数说明:

void * addr: 必须是前面某一次 ldm_malloc 返回的分配成功的局存地址;

size_t size: 显式指定释放空间的大小，一般必须与前面某一次 ldm_malloc 的大小相匹配。

返回值:

无

第四章 加速线程库

加速线程库(Athread 库)是针对主从加速编程模型所设计的程序加速库，其目的是为了用户能够方便、快捷地对核组内的线程进行控制和调度，从而更好地发挥核组内多从核执行加速性能。

一般来说并发线程的应用程序无需考虑可用处理器的数量，但是在本编译系统面向的众核架构下，每个线程绑定一个从核资源，所以在使用 Athread 库的时候，需要考虑当前从核资源的分配状况。

4.1 主核加速线程库

主核加速线程库主要是提供主核程序使用的 Athread 接口，主要用于控制线程的创建回收、线程调度控制等一系列操作。用户可以通过调用以下接口实现相关功能，使用方法：`#include <athread.h>`。

4.1.1 初始化线程库

函数名：

`int athread_init()`

函数说明：

完成加速线程库的初始化，在使用任何非大共享模式的主核线程库接口前必须调用该接口。

参数说明：

无

返回值：

成功：返回 0，表示加速线程初始化成功；

失败：其它任何返回值都表示初始化失败。

4.1.2 创建单线程

函数名：

`int athread_create(int id, void* fpc, void * arg)`

函数说明：

在当前进程中添加新的受控线程。线程执行的起始任务由函数 fpc 指定；函数 fpc 的参数由 arg 提供。

参数说明：

int id: 线程绑定 ID 号；

start_routine fpc: 函数指针；

void * arg: 函数 fpc 的参数起始地址。

返回值：

成功：返回 0。

失败：其它任何返回值都表示出现了错，如果监测到以下任一情况，`athread_create` 将失败，并返回相应的值。

- `-EINVAL`: ID 号非法(负数，或者超过最大线程数)；或者 ID 号已经被占用，被占资源没有释放。

附加说明：

线程绑定 ID 号后，线程位置跟 ID 号无关。

4.1.3 等待单线程终止

函数名：

`int athread_wait (int id)`

函数说明：

显式阻塞调用该线程，直到指定的线程终止。指定的线程必须位于当前的进程中。

参数说明：

int id: 等待线程 ID 号

返回值：

成功：返回 0；

失败：其它任何返回值都表示出现了错；如果监测到以下任一情况，`athread_wait` 将失败，并返回相应的值。

- `-EINVAL`: ID 号非法或没有找到指定线程 ID 对应的线程。

4.1.4 关闭单线程流水线

函数名：

`int athread_end (int id)`

函数说明：

在确定线程所占从核无相关作业后，停滞从核流水线，关闭从核，该从核在本进程中将无法再次使用。

参数说明：

int id: 指定线程 ID 号

返回值：

成功：返回 0，线程所占用的从核资源被成功关闭。

失败：其它任何返回值都表示出现了错，如果监测到以下任一情况，`athread_end` 将失败，并返回相应的值。

- `-EINVAL`: 线程从核流水线终止失败，ID 对应线程仍然在作业运行中；没有找到给定线程 ID 对应的线程。

附加说明：

必须保证从核无任何用户作业才能使用该函数。

4.1.5 创建线程组（带 Cache 刷新）

函数名：

`int pthread_spawn(void* fpc, void * arg)`

函数说明：

在当前进程中添加新的受控线程组。线程执行的任务由函数 `fpc` 指定；函数 `fpc` 的参数由 `arg` 提供。

参数说明：

`void * fpc`: 函数指针；

`void * arg`: 函数 `fpc` 的参数起始地址。

返回值：

成功：返回 0。

失败：其它任何返回值都表示出现了错，如果监测到以下情况，`pthread_spawn` 将失败，并返回相应的值。

- `-EINVAL`: 线程组创建失败。

附加说明：

调用 `pthread_spawn` 接口时，启动核组中的所有可用从核资源。使用 `pthread_spawn` 接口启动的从核函数，从核函数会在运行前后自动将本从核整个 Cache 刷新到主存。

4.1.6 创建线程组（不带 Cache 刷新）

函数名：

`int pthread_spawn_noflush(void* fpc, void * arg)`

函数说明：

在当前进程中添加新的受控线程组。线程执行的任务由函数 `fpc` 指定；函数 `fpc` 的参数由 `arg` 提供。

参数说明：

`void * fpc`: 函数指针；

`void * arg`: 函数 `fpc` 的参数起始地址。

返回值：

成功：返回 0。

失败：其它任何返回值都表示出现了错，如果监测到以下情况，`pthread_spawn_noflush` 将失败，并返回相应的值。

- `-EINVAL`: 线程组创建失败。

附加说明：

调用时 `pthread_spawn_noflush` 接口时，启动核组中的所有可用从核资源。

`pthread_spawn_noflush` 接口同样用于启动从核函数，但不会在从核函数运行结束后自动将本从核整个 Cache 刷新到主存。

4.1.7 等待线程组终止

函数名:

`int athread_join ()`

函数说明:

显式阻塞等待该线程组，直到指定的线程组终止。指定的线程组必须位于当前的进程中。

参数说明:

无

返回值:

成功: 返回 0;

失败: 其它任何返回值都表示出现了错，如果监测到以下情况，`athread_join` 将失败，并返回相应的值。

- `-EINVAL`: 核组内无线程组运行。

4.1.8 关闭线程组流水线

函数名:

`int athread_halt ()`

函数说明:

在确定线程组所占所有从核无相关作业后，停滞从核组流水线，关闭从核组，该从核组在本进程无法再次使用。

参数说明:

无

返回值:

成功: 返回 0，线程所占用的从核资源被成功关闭。

失败: 其它任何返回值都表示出现错误，如果监测到以下情况，`athread_halt` 将失败，并返回相应的值。

- `-EINVAL`: 无法正常关闭从核资源。

附加说明:

必须保证从核组无任何用户作业才能使用该函数。

4.1.9 快速线程组: 进入快速线程组模式

函数名:

`int athread_enter64 ()`

函数说明:

在使用 `athread_spawn64` 之前，调用 `athread_enter64` 函数进行初始化（在此之前仍然需要调用 `athread_init` 接口）。

参数说明：

无

返回值：

成功：返回 0。

失败：其它任何返回值都表示出现了错。

附加说明：

调用 `athread_enter64` 接口时，必须保证核组中的所有 64 从核资源可用。

4.1.10 快速线程组：创建线程组

函数名：

`int athread_spawn64 (void* fpc)`

函数说明：

在当前进程中快速添加新的受控 64 从核线程组。线程执行的任务由函数 `fpc` 指定。

参数说明：

`void * fpc`: 函数指针

返回值：

成功：返回 0。

失败：其它任何返回值都表示出现了错。

附加说明：

调用 `athread_spawn64` 接口时，必须在调用 `athread_enter64` 接口之后，必须保证核组中的所有 64 从核资源可用。

4.1.11 快速线程组：等待线程组终止

函数名：

`int athread_join64 ()`

函数说明：

显式阻塞等待 64 从核线程组，直到指定的线程组终止。

参数说明：

无

返回值：

成功：返回 0；

失败：其它任何返回值都表示出现了错。

附加说明：

调用 `athread_join64` 接口时，必须和 `athread_spawn64` 配对使用，必须保证核组中的所有 64 从核资源可用。

4.1.12 快速线程组：离开快速线程组模式

函数名：

`int athread_leave64 ()`

函数说明：

在确定不再使用 `athread_spawn64` 启动快速线程组时，使用该函数离开快速线程组模式。

参数说明：

无

返回值：

成功：返回 0。

失败：其它任何返回值都表示出现了错。

附加说明：

调用 `athread_leave64` 接口时，必须和 `athread_enter64` 配对使用，必须保证核组中的所有 64 从核资源可用。

4.1.13 快速线程组：进入带参数快速线程组模式

函数名：

`int athread_enter64_arg ()`

函数说明：

在使用 `athread_spawn64_arg` 之前，调用 `athread_enter64_arg` 函数进行初始化（在此之前仍然需要调用 `athread_init` 接口）。

参数说明：

无

返回值：

成功：返回 0。

失败：其它任何返回值都表示出现了错。

附加说明：

调用 `athread_enter64_arg` 接口时，必须保证核组中的所有 64 从核资源可用。

4.1.14 快速线程组：创建带参数线程组

函数名：

`int athread_spawn64_arg (void* fpc , void* arg)`

函数说明：

在当前进程中快速添加新的受控 64 从核线程组。线程执行的任务由函数 `fpc` 指定。

参数说明：

`void * fpc`: 函数指针

void * arg: 函数 fpc 的参数起始地址。

返回值:

成功: 返回 0。

失败: 其它任何返回值都表示出现了错。

附加说明:

调用 `athread_spawn64_arg` 接口时, 必须在调用 `athread_enter64_arg` 接口之后, 必须保证核组中的所有 64 从核资源可用。

4.1.15 快速线程组: 等待带参数线程组终止

函数名:

`int athread_join64_arg ()`

函数说明:

显式阻塞等待 64 从核线程组, 直到指定的线程组终止。

参数说明:

无

返回值:

成功: 返回 0;

失败: 其它任何返回值都表示出现了错。

附加说明:

调用 `athread_join64_arg` 接口时, 必须和 `athread_spawn64_arg` 配对使用, 必须保证核组中的所有 64 从核资源可用。

4.1.16 快速线程组: 离开带参数快速线程组模式

函数名:

`int athread_leave64_arg ()`

函数说明:

在确定不再使用 `athread_spawn64_arg` 启动快速线程组时, 使用该函数离开快速线程组模式。

参数说明:

无

返回值:

成功: 返回 0。

失败: 其它任何返回值都表示出现了错。

附加说明:

调用 `athread_leave64_arg` 接口时, 必须和 `athread_enter64_arg` 配对使用, 必须保证核组中的所有 64 从核资源可用。

4.1.17 大共享模式：初始化线程库

函数名：

`int athread_init_all()`

函数说明：

完成加速线程库的初始化，在使用任何大共享模式的线程库接口前必须调用该接口。

参数说明：

无

返回值：

成功：返回 0，表示加速线程初始化成功；

失败：其它任何返回值都表示出现了错。

附加说明：

该接口会初始化全片 6 核组的相关资源。

要使用大共享模式的线程库接口，运行时需传递加载器参数“-a 1”。该模式下 1 个进程会占用全片核组资源，如全片资源不满可能会出现异常。

4.1.18 大共享模式：创建线程组

函数名：

`int athread_spawn_all(int cgmask, void* fpc, void * arg)`

函数说明：

在当前进程中添加新的受控线程组。启动核组位图由 cgmask 指定，线程执行的任务由函数 fpc 指定；函数 fpc 的参数由 arg 提供。

参数说明：

int cgmask: 核组位图 (0x1 代表 0 核组, 0x2 代表 1 核组, 0x3f 代表全核组.....);

void * fpc: 函数指针;

void * arg: 函数 fpc 的参数起始地址。

返回值：

成功：返回 0。

失败：其它任何返回值都表示出现了错，如果监测到以下情况，`athread_spawn_all` 将失败，并返回相应的值。

- -EINVAL: 线程组创建失败。

附加说明：

调用时 `athread_spawn_all` 接口时，启动核组中的所有可用从核资源。

使用 `athread_spawn_all` 接口启动的从核函数，从核函数会在运行结束后自动将本从核整个 Cache 刷新到主存。

要使用大共享模式的线程库接口，运行时需传递加载器参数“-a 1”。该模式下 1 个进程会占用全片核组资源，如全片资源不满可能会出现异常。

4.1.19 大共享模式：等待线程组终止

函数名：

`int athread_join_cgmask (int cgmask)`

函数说明：

显式阻塞等待指定 CG 的线程组，直到该线程组终止。指定的线程组必须位于当前的进程中。

参数说明：

int cgmask: 核组位图（0x1 代表 0 核组，0x2 代表 1 核组，0x3f 代表全核组……）

返回值：

成功：返回 0；

失败：其它任何返回值都表示出现了错，如果监测到以下情况，`athread_join_cgmask` 将失败，并返回相应的值。

- -EINVAL: 核组内无线程组运行。

附加说明：

要使用大共享模式的线程库接口，运行时需传递加载器参数“-a 1”。该模式下 1 个进程会占用全片核组资源，如全片资源不满可能会出现异常。

4.1.20 大共享模式：等待所有线程组终止

函数名：

`int athread_join_all ()`

函数说明：

显式阻塞等待全片 6CG 的线程组，直到所有线程组终止。指定的线程组必须位于当前的进程中。

参数说明：

无。

返回值：

成功：返回 0；

失败：其它任何返回值都表示出现了错，如果监测到以下情况，`athread_join_all` 将失败，并返回相应的值。

- -EINVAL: 核组内无线程组运行。

附加说明：

要使用大共享模式的线程库接口，运行时需传递加载器参数“-a 1”。该模式下 1 个进程会占用全片核组资源，如全片资源不满可能会出现异常。

4.1.21 大共享模式：绑定线程到指定核组

函数名：

`int SET_TO_SWCG (int cgid)`

函数说明：

将线程函数绑定到指定 CG 上运行。

参数说明：

int cgid: CG 号，合法值为 0-5。

返回值：

成功：返回 0；

失败：其它任何返回值都表示出现了错。

附加说明：

要使用大共享模式的线程库接口，运行时需传递加载器参数“-a 1”。该模式下 1 个进程会占用全片核组资源，如全片资源不满可能会出现异常。

4.1.22 设置并行区线程总数

函数名：

`int athread_set_num_threads (int num)`

函数说明：

使用该接口设置当前进程下一个并行区启动的所有核组线程总数。

参数说明：

int num: 线程总数，范围是[0,64]。

返回值：

成功：0；

失败：其它任何返回值都表示出现了错，如果监测到以下任一情况，`athread_set_num_threads` 将失败，并返回相应的值。

- -EINVAL: 并行区线程数设置失败，比如上个并行区任务未完成；设置线程数目超过了可用线程资源。

4.1.23 获取并行区线程总数

函数名：

`int athread_get_num_threads ()`

函数说明：

使用该接口得到当前进程启动的所有核组线程总数。

参数说明：

无

返回值：

成功：返回当前核组内并行区可用线程个数($1 \leq \text{ret} \leq 64$)；

失败：其它任何返回值都表示出现了错。

4.1.24 获取主核核组号

函数名:

`int current_array_id()`

函数说明:

使用该接口得到当前主核核组号。

参数说明:

无

返回值:

成功: 返回当前主核核组号, 合法值为 0-5;

失败: 其它任何返回值都表示出现了错。

4.2 从核加速线程库

从核加速线程库主要是提供从核程序的 Athread 接口, 主要包括从核 Cache 和 LDM 的相关配置、从核 Cache 的刷新和淘汰、主存和 LDM 间的 DMA 内部接口、从核阵列内的同步等操作, 使用方法: `#include <slave.h>`。而全片的同步、从核间的 RMA 消息、锁、DMA 操作的用户层接口等在运行时系统中实现, 用户可查阅相关手册。

4.2.1 数据传输 DMA 内部接口

函数名:

`__builtin_sw_slave_athread_dma (dma_mode mode, DMA_OP op, void *addr, void *ldm_addr, int len, void *reply, int bcast_mask, int stride, int bsize)`

Fortran 函数名:

`athread_dma`

函数参数与 C 函数原型一致

函数说明:

从核局存 LDM 接收主存 MEM 数据, 进行主存 MEM 到从核 LDM 的数据 get 操作, 将 MEM 的数据 get 到 LDM 指定位置; 或者从核局存 LDM 往主存 MEM 发送数据, 进行从核 LDM 到主存 MEM 的数据 put 操作, 将 LDM 的数据 put 到 MEM 指定的位置。传输模式由 mode 指定, 传输方向由 op 指定。

参数说明:

`dma_mode mode`: DMA 传输命令模式;

`DMA_OP op`: DMA 传输命令操作码;

`void *addr`: DMA 传输主存地址;

`void *ldm_addr`: DMA 传输本地局存地址;

`int len`: DMA 传输数据量, 以字节为单位;

`void *reply`: DMA 传输回答字地址, 必须为局存地址, 地址 4B 对界;

`int bcast_mask`: DMA 传输多播使能, 列多播时表示参与多播的列, 某 bit 为 1 表示

对应的列参加多播，仅支持同簇内列多播；

int stride: 主存跨步，以字节为单位；

int bsize: 跨步向量块大小，DMA 跨步传输时有效，表示 DMA 传输的跨步向量块大小，以字节为单位。

返回值:

- 无

附加说明:

```
typedef enum {
    PE_MODE,    //单从核模式
    COL_MODE=2  //列多播模式
} dma_mode;
typedef enum {
    DMA_PUT,
    DMA_GET,
    DMA_PUT_P, (物理地址)
    DMA_GET_P, (物理地址)
    DMA_BARRIER = 5,
    ALL_BARRIER = 6
} DMA_OP;
```

举例说明:

```
__thread_local long ldm_a[64];
extern long mem_a[64];
volatile __thread_local int reply=0; //需声明成 volatile
__builtin_sw_slave_athread_dma(PE_MODE,DMA_GET,mem_a,ldm_a,64*8,&reply,0,0); //以单从核模式，主存以 mem_a 为基地址取 64*8B 数据放入从核局存以 ldm_a 为基地址的连续 64*8B 的空间中，其中多播使能、主存跨步、跨步向量块大小均为 0。
while(reply==0); //等待 dma 传输完成
__builtin_sw_slave_athread_dma(PE_MODE,DMA_PUT,mem_a,ldm_a,64*8,&reply,0,0); //以单从核模式，从核局存以 ldm_a 为基地址取连续 64*8B 数据存入以 mem_a 为基地址的连续 64*8B 空间中，其中多播使能、主存跨步、跨步向量块大小均为 0。
while(reply==0); //等待 dma 传输完成
```

备注:

编译时需加-msimd 选项

Fortran 举例说明:

```
module modx
real*8 mem_a
end
```

```

module ldm__modr !in ldm space
integer, volatile :: reply
end

subroutine fun(args)
use modx
use ldm__modr
real*8 ldm_a
common /local_a/ ldm_a
call athread_dma(0,1,mem_a,ldm_a,8,reply,0,0,0) !dma_get
do while (reply .eq. 0)
end do
print *, ldm_a
ldm_a=3
reply=0
call athread_dma(0,0,mem_a,ldm_a,8,reply,0,0,0) !dma_put
do while (reply .eq. 0)
end do
end

```

备注：

- 1、编译时需加-msimd 选项
- 2、volatile 与 common 属性有冲突，reply 可以安照本例方式使用，编译加-fmodule-ldm 选项；也可以作为栈变量声明，bsub 提交作业时加-b 选项指定从核栈使用 ldm 空间。

4.2.2 获取从核连续共享 LDM 大小

函数原型：

`long get_slave_ldmshare_size ( )`

函数说明：

得到当前从核的连续共享 LDM 空间大小

参数说明：

无

返回值：

返回值单位为字节。有 7 种可能的返回值：分别为 0、4096、8192、16384、32768、65536、131072，即代表连续共享 LDM 大小为 0、4KB、8KB、16KB、32KB、64KB、128KB。

附加说明：

也可在从核程序中直接使用_ldm_share_size 局存变量获得连续共享 LDM 空间大小。

4.2.3 获取从核 Cache 大小

函数名:

`long get_slave_cache_size()`

函数说明:

得到当前从核 Cache 空间大小。

参数说明:

无

返回值:

返回值单位为字节。有 3 种可能的返回值: 分别为 0、32768、131072, 即代表 Cache 大小为 0、32KB、128KB。

附加说明:

也可在从核程序中直接使用 `_cache_size` 局存变量获得连续从核 Cache 空间大小。

4.2.4 配置从核连续共享 LDM 模式与大小

函数名:

`int set_ldm_mode_size(int ldm_shared_mode, int ldm_shared_size)`

函数说明:

配置从核的连续共享 LDM 模式及大小。

参数说明:

`ldm_shared_mode`: 从核连续共享 LDM 模式, 支持:

<code>LDMS_SINGLE</code>	单簇
<code>LDMS_DOUBLE</code>	双簇
<code>LDMS_QUAD</code>	四簇
<code>LDMS_ALL_CLUSTER</code>	全阵列簇
<code>LDMS_ALL_ROW</code>	全阵列行

`ldm_shared_size`: 从核连续共享 LDM 大小, 支持:

`LDMS_0KB | LDMS_4KB | LDMS_8KB | LDMS_16KB |`
`LDMS_32KB | LDMS_64KB | LDMS_128KB`

返回值:

0 成功; 1 失败。

附加说明:

- 1、硬件要求所有从核的连续共享 LDM 模式以及容量必须一致, 这一点需要用户在配置时保证;
- 2、为保证程序运行的正确性, 一般用户对连续共享 LDM 模式及容量以及下一节的 Cache 容量进行配置时, 应当遵循以下的流程:
 - a) 通过判回答字, 确认从核之前参与的 DMA/RMA 请求都已完成;
 - b) 插入 `memb` 指令, 确认从核之前发出的访存操作都已完成;
 - c) 进行核组内从核的全同步;

- d) 调用配置接口进行配置;
- e) 再次进行核组内从核的全同步;
- 3、上述流程会带来较大的时间开销,对性能要求比较高的用户在能够熟练掌握连续共享 LDM 以及 Cache 运行使用机制的基础上,如果能够保证多个从核的运行速度基本同步的情况下,可以省却上面的 MEMB 指令以及从核同步的步骤。
- 4、-b 选项用于将从核程序栈指针指向 LDM 空间,这样可以使对栈变量的访问转为对 LDM 空间上的访问,提升程序性能。需要注意的是,如果栈空间太大,程序运行过程中可能会对原有用户的 LDM 空间的使用造成影响,产生不可预知的错误。尤其是动态调整从核连续共享 LDM 模式/容量以及从核 cache 容量时,会额外动态申请与栈空间大小相同的 LDM 空间进行栈空间的转移工作。

4.2.5 配置从核 Cache 容量

函数名:

`int set_cache_size (int cache_size)`

函数说明:

配置从核的 Cache 容量。

参数说明:

cache_size: 从核 Cache 大小, 支持:
CACHE_0KB | CACHE_32KB | CACHE_128KB

返回值:

0 成功; 1 失败。

附加说明:

- 1、调整从核 Cache 容量过程中会对整个从核 cache 进行刷新,用户需要注意数据的一致性問題。
- 2、一般情况下要求所有从核的 Cache 容量必须一致。如果用户不配置连续共享 LDM 空间,允许针对不同的从核配置不同的 cache 容量,一旦配置了连续共享 LDM 空间,则必须保证所有从核的 Cache 容量相同。这一点需要用户在配置时保证。
- 3、为保证程序运行的正确性,用户配置遵循的流程与 4.2.4 节相同。
- 4、-b 选项用于将从核程序栈指针指向 LDM 空间,这样可以使对栈变量的访问转为对 LDM 空间上的访问,提升程序性能。需要注意的是,如果栈空间太大,程序运行过程中可能会对原有用户的 LDM 空间的使用造成影响,产生不可预知的错误。尤其是动态调整从核连续共享 LDM 模式/容量以及从核 cache 容量时,会额外动态申请与栈空间大小相同的 LDM 空间进行栈空间的转移工作。

4.2.6 淘汰连续主存空间对应 Cache 行

函数名:

`void evict_slave_cache_cont (void* start, void* end)`

函数说明：

淘汰连续主存空间 [start, end] 对应 Cache 行。

参数说明：

void * start: 主存空间起始地址

void * end: 主存空间结束地址

返回值：

无

附加说明：

start 和 end 为主存空间地址，与数据类型无关。对于结构体数组，需要用户自行保证起始地址与结束地址的正确性。

4.2.7 刷新整个从核 Cache

函数名：

`void flush_slave_cache ()`

函数说明：

刷新整个从核 Cache。

参数说明：

无

返回值：

无

4.2.8 从核 Cache 写暗示

函数名：

`void sw_slave_write_hint (void * mem)`

函数说明：

将主存地址 mem 对应的 Cache 行置写暗示。

参数说明：

void * mem: 主存地址。

返回值：

无

附加说明：

写暗示指令提示包含寻址字节的对界 256 字节块的内容将不再有读访问，但不久将有写操作。从核将不读存储器以前的内容而直接分配并占有该 Cache 行，Cache 行的内容可设为任意值。写暗示避免从主存中载入无用的数据，提高了处理性能。

4.2.9 从核 Cache 私有段设置

函数名：

```
void sw_slave_private_section0 (void * start, void * end)
void sw_slave_private_section1 (void * start, void * end)
void sw_slave_private_section2 (void * start, void * end)
void sw_slave_private_section3 (void * start, void * end)
```

函数说明：

将主存空间 [start, end] 对应 Cache 行设置为私有段，目前支持 4 套私有空间的配置。

参数说明：

void * start: 主存空间起始地址；
void * end: 主存空间结束地址。

返回值：

无

附加说明：

映射到该私有段区间内的 Cache 行写回主存时以独占方式写回，用于改善写回主存性能。

start 和 end 为主存空间地址，与数据类型无关。对于结构体数组，需要用户自行保证起始地址与结束地址的正确性。

4.2.10 从核阵列内同步

函数名：

```
int pthread_syn (scope scp, int mask)
```

函数说明：

单核组从核阵列内同步的三种模式：行同步、列同步与点同步

参数说明：

scope scp: 从核同步模式，支持：

ROW_SCOPE	行同步模式
COL_SCOPE	列同步模式
PP_SCOPE	点同步模式

int mask: 位图参数。

行同步、列同步时，mask 合法范围为 0x1 - 0xff，寄存器中低 8 位有效，[7:0]分别指示同行（或同列）从核 7 到从核 0 是否同步，为 1 时指示该从核参与同步，为 0 时屏蔽该从核。点同步时，mask 合法范围为 0-63，表示与当前从核同步的对应从核号。

返回值：

成功：返回 0。

失败：其它任何返回值都表示出现了错。

附加说明：

同步时，进行同步的从核号必须包含在同步向量中，否则产生异常：slave core was not included in the sync mask。

4.2.11 从核阵列内全同步

函数名:

`void athread_syn_all()`

函数说明:

单核组从核阵列内全同步，同步该核组 64 个从核。

参数说明:

无

返回值:

无

4.2.12 从核指针设成不可 cache 访问

函数名:

`void * set_ptr_uncached(void *addr);`

函数说明:

将指针 addr 指向的地址设为不可 cache 访问。

参数说明:

`void *addr`

返回值:

无

4.2.13 从核指针设成可 cache 访问

函数名:

`void * set_ptr_cached(void *addr);`

函数说明:

将指针 addr 指向的地址设为可 cache 访问。

参数说明:

`void *addr`

返回值:

无

示例:

```
#include<stdio.h>
```

```
int a[100]; //可 cache 空间
```

```
void main()
```

```
{
```

```
    int *p1=set_ptr_uncached(a); //通过 p1 访问数组 a 为不可 cache 访问
```

```

        int *p2=set_ptr_cached(p1); //通过 p2 访问数组 a 为可 cache 访问
        printf("%p,%p,%p\n",a,p1,p2);
    }

```

4.2.14 从核半精度浮点 FP16 格式和 BF16 格式切换

从核半精度浮点支持 FP16 格式和 BF16 格式，FP16 格式精度高数值范围小，BF16 格式精度低数值范围大，两种格式各有优缺点，单纯使用一个格式可能满足不了课题需求，从核加速线程库提供了半精度浮点格式切换接口和数据转换接口，支持两种半精度浮点格式的切换，满足特定场景的需求。

半精度浮点格式切换接口包括 `set_fp16_mode` 和 `set_bf16_mode`，使用时需包含 `slave.h` 头文件，编译器默认使用 FP16 格式，编译和链接时加 `-mslave-bf16` 选项时使用 BF16 格式，课题中从核程序调用 `set_fp16_mode` 将本从核线程切换为 FP16 格式，调用 `set_bf16_mode` 将本从核线程切换为 BF16 格式。

半精度浮点数据转换接口包括 `fcvthh`、`convert_float16`、`vfcvthh`、`simd_convert_float16` 接口，`fcvthh`、`convert_float16` 使用标量类型，使用时包含 `slave.h` 头文件；`vfcvthh`、`simd_convert_float16` 使用向量类型，使用时包含 `simd.h` 头文件。

函数名：

```
float16 fcvthh(float16 __a)
```

函数说明：

将标量输入参数 `__a` 由当前半精度浮点格式转换成另一种格式后返回输出。

参数说明：

```
float16 __a
```

返回值：

```
float16
```

函数名：

```
void convert_float16(float16 *dest, float16 *src, int num)
```

函数说明：

循环调用 `fcvthh` 接口，将 `src` 地址开始的 `num` 个半精度浮点数由当前半精度浮点格式转换成另一种格式，写入 `dest` 地址段。

参数说明：

```
float16 *dest 目的地址
```

```
float16 *src 源地址
```

```
int num 转换的半精度浮点数据个数
```

返回值：

无

函数名:

float16v32 vfcvthh(float16v32 __a)

函数说明:

将向量输入参数__a 由当前半精度浮点格式转换成另一种格式后返回输出。

参数说明:

float16v32 __a

返回值:

float16v32

函数名:

void simd_convert_float16(float16 *dest, float16 *src, int num)

函数说明:

循环调用 vfcvthh 接口，将 src 地址开始的 num 个半精度浮点数由当前半精度浮点格式转换成另一种格式，写入 dest 地址段。

参数说明:

float16 *dest 目的地址，要求 64 字节对界

float16 *src 源地址，要求 64 字节对界

int num 转换的半精度浮点数据个数，要求是 32 的倍数

返回值:

无

半精度浮点数据类型转换接口是将数据由当前格式转换成另一种格式，因此在调用格式切换接口前需要把数据先转换好。

立即数是在编译时处理的，程序中进行格式切换时立即数的值就不对了，也需要调用转换接口进行转换。

标量示例: cvt.c

```
#include<slave.h>
```

```
__thread_local float16 a=1.0,b;
```

```
main()
```

```
{
```

```
    printf("before slave%d:%x\n",_PEN,*(unsigned short *)&a);
```

```
    float16 x=0.5;
```

```
    if(_PEN==0 || _PEN==1){
```

```
        x=fcvthh(x);
```

```
        a=fcvthh(a);
```

```
    }
```

```
    if(_PEN==0 || _PEN==2)
```

```
        set_bf16_mode();
```

```

        b=a+x;
        printf("after slave%d:%x\n",_PEN,*(unsigned short *)&a);
        printf("after slave%d:a=%f,x=%f,b=%f\n",_PEN,(float)a,(float)x,(float)b);
    }

```

编译: swgcc -mslave cvt.c

提交 3 个从核运行结果为:

before slave0:0x3c00

before slave1:0x3c00

before slave2:0x3c00

after slave0:0x3f80

after slave1:0x3f80

after slave2:0x3c00

after slave0:a=1.000000,x=0.500000,b=1.500000

after slave1:a=1.875000,x=1.750000,b=3.625000

after slave2:a=0.007812,x=0.000031,b=0.007812

0 号从核进行了数据转换和格式切换, 结果对; 1 号从核只进行了数据转换, 2 号从核只进行了格式切换, 结果都不对。

标量示例: vcvt.c

```
#include<slave.h>
```

```
#include<simd.h>
```

```
#define N 64
```

```
float16 a[N],b[N] __attribute__((aligned(64)));
```

```
main()
```

```
{
```

```
    int i;
```

```
    for(i=0;i<N;i++)
```

```
        a[i]=i;
```

```
    simd_convert_float16(b,a,N);
```

```
    set_bf16_mode();
```

```
    for(i=0;i<N;i++)
```

```
        printf("%f\n",(float)b[i]);
```

```
}
```

编译: swgcc -mslave -msimd vcvt.c

运行打印 0.0~63.0

4.3 主核 IO 空间操作相关接口

使用方法: #include<athread.h>

4.3.1 SPEIO_REG_ADDR

函数原型:

`unsigned long * SPEIO_REG_ADDR (int cgid, int speid, unsigned long reg)`

函数说明:

获取指定从核的内部寄存器 IO 访问地址。

参数说明:

cgid 核组号 (0-5)
speid 从核号 (0-63)
reg 从核内部寄存器的 IO 地址

预定义的 reg 包括有:

#define SPE_EBXIO_FPCR	0x100800	//浮点控制状态寄存器
#define SPE_EBXIO_DEF0	0x100400	//用户自定义项 0
#define SPE_EBXIO_CTRL	0x100900	//EBOX 控制寄存器
#define SPE_IBXIO_GPC	0x8000	//IBOX 全局 PC
#define SPE_IBXIO_RUN	0x8100	//IBOX 运行控制

返回值:

从核内部寄存器 IO 访问地址。

4.3.2 SPEIO_READ_REG

函数原型:

`unsigned long SPEIO_READ_REG (int cgid, int speid, unsigned long reg)`

函数说明:

通过 IO 访问获取指定从核的内部寄存器内容。

参数说明:

cgid 核组号 (0-5)
speid 从核号 (0-63)
reg 从核内部寄存器的 IO 地址

预定义的 reg 包括有:

#define SPE_EBXIO_FPCR	0x100800	//浮点控制状态寄存器
#define SPE_EBXIO_DEF0	0x100400	//用户自定义项 0
#define SPE_EBXIO_CTRL	0x100900	//EBOX 控制寄存器
#define SPE_IBXIO_GPC	0x8000	//IBOX 全局 PC
#define SPE_IBXIO_RUN	0x8100	//IBOX 运行控制

返回值:

从核内部寄存器的 64 位内容。

4.3.3 SPEIO_WRITE_REG

函数原型:

```
void SPEIO_WRITE_REG (int cgid, int speid, unsigned long reg, unsigned long val)
```

函数说明:

通过 IO 访问写指定从核的内部寄存器.

参数说明:

cgid 核组号 (0-5)
speid 从核号 (0-63)
reg 从核内部寄存器的 IO 地址
val 要写入内部寄存器的值

预定义的 reg 包括有:

#define SPE_EBXIO_FPCR	0x100800	//浮点控制状态寄存器
#define SPE_EBXIO_DEF0	0x100400	//用户自定义项 0
#define SPE_EBXIO_CTRL	0x100900	//EBOX 控制寄存器
#define SPE_IBXIO_GPC	0x8000	//IBOX 全局 PC
#define SPE_IBXIO_RUN	0x8100	//IBOX 运行控制

返回值:

无

4.3.4 SPEIO_LDM_ADDR

函数原型:

```
unsigned long * SPEIO_LDM_ADDR (int cgid, int speid, unsigned long disp)
```

函数说明:

获取指定从核 LDM 的 IO 访存地址。

参数说明:

cgid 核组号 (0-5)
speid 从核号 (0-63)
disp 从核内部 LDM 地址 (0~256KB), 至少 4 字节对界且需要保证使用类型对界要求, 如 int 要 4 字节对界, double 要 8 字节对界。

返回值:

指定从核的 LDM 的 IO 访存地址。

附加说明:

跨核组访问从核 LDM 的 IO 访存地址时, 需要调用核心接口获取权限。

示例:

```
#include<athread.h>
```

```

#include<crts.h>
extern void slave_fun();
main()
{
    athread_init();
    CRTS_init();
    int cgid=current_array_id();
    if(cgid==0)
    {
        int stask_fd=athread_get_slave_fd();
        int val=ioctl(stask_fd, SLAVE_MAP_SPE_IO, 1); //获取 1 核组 IO 权限
        if(val != 0) {
            printf("map stask spe failed %d, %d\n",val,errno);
        }
        unsigned long destptr = (void *)SPEIO_LDM_ADDR(1,1,32); //写 1 核组 1 从
        //核地址为 32 的 LDM 空间
        *(long *)destptr=2;
    }
    CRTS_sync_node (); //主核同步等待 IO 操作完成
    athread_spawn(fun,0);
    athread_join();
    athread_halt();
}

```

4.4 浮点控制状态寄存器 fpcr 配置接口

4.4.1 主核读写 fpcr 相关接口

使用方法：#include<athread.h>

4.4.1.1 RFPCR

函数原型：

`void RFPCR (unsigned long val)`

函数说明：

读主核浮点控制寄存器 fpcr，将寄存器值存到 val 变量中。

参数说明：

val：用于存储 fpcr 寄存器值的变量。

返回值：

无

4.4.1.2 WFPCR

函数原型:

`void WFPCR(unsigned long val)`

函数说明:

将 val 变量值写到主核浮点控制寄存器 fpcr。

参数说明:

val: 用于写入 fpcr 寄存器的值。

返回值:

无

4.4.2 从核读写 fpcr 相关接口

使用方法: `#include<slave.h>`

4.4.2.1 RFPCR0

函数原型:

`void RFPCR0 (unsigned long val)`

函数说明:

读从核浮点控制寄存器 fpcr 低 64 位值到 val 变量。

参数说明:

val: 用于存储 fpcr 寄存器低 64 位值的变量。

返回值:

无

4.4.2.2 RFPCR1

函数原型:

`void RFPCR1 (unsigned long val)`

函数说明:

读从核浮点控制寄存器 fpcr 高 64 位值到 val 变量。

参数说明:

val: 用于存储 fpcr 寄存器高 64 位值的变量。

返回值:

无

4.4.2.3 RFPCR

函数原型:

`void RFPCR (unsigned long val0, unsigned long val1)`

函数说明:

读从核浮点控制寄存器 fpcr，将低 64 位值存到 val0 变量，高 64 位值存到 val1 变量。

参数说明:

val0 用于存储 fpcr 寄存器低 64 位值的变量

val1 用于存储 fpcr 寄存器高 64 位值的变量

返回值:

无

4.4.2.4 WFPCR

函数原型:

`void WFPCR (unsigned long val0, unsigned long val1)`

函数说明:

将 val0、val1 变量拼成 128 位值写到从核浮点控制寄存器，val0 为低 64 位值，val1 为高 64 位值。

参数说明:

val0 用于写 fpcr 寄存器低 64 位值的变量

val1 用于写 fpcr 寄存器高 64 位值的变量

返回值:

无

4.4.3 设置浮点运算舍入模式相关接口

使用方法:

主核: `#include <athread.h>`

从核: `#include <slave.h>`

函数原型:

`void SET_round_mode (int val)`

函数说明:

将浮点控制寄存器 fpcr 中的动态舍入位设置为用户定义的值。

参数说明:

val: 舍入模式，有效值只有 0、1、2、3 四个值，其他无效且会报错;

0: 向“0”舍入

1: 向负无穷大舍入

- 2: 就近舍入
- 3: 向正无穷大舍入

返回值:

无

4.5 加速线程库示例

4.5.1 加速线程库的示例：创建单线程

[host.c]

```
#include <athread.h>

extern void SLAVE_FUN(fun(void * arg));

int main(){
    int arg1[2],arg2[2];
    athread_init(); //线程库初始化
    arg1[0]=1;
    arg1[1]=2;
    athread_create(0, fun,arg1);//创建绑定线程，线程 ID 号为 0
    arg2[0]=3;
    arg2[1]=4;
    athread_create(1, fun,arg2);//创建绑定线程，线程 ID 号为 1
    athread_wait(0);//等待线程结束
    athread_wait(1);//等待线程结束
    //关闭线程所占从核流水线，该从核无法在本进程中继续使用
    athread_end(0);
    athread_end(1);
}
```

[slave.c]

```
#include <slave.h>
#include <stdio.h>
#include <string.h>

void fun(int * arg){
    char str[2048];
    int result = *arg + *(arg+1);
    sprintf(str,"now in %d, sum=%d\n",_PEN,result);
    write(1,str,strlen(str));
}
```

4.5.2 加速线程库的示例：单核组

[host.c]

```
#include<pthread.h>
extern void SLAVE_FUN(fun());
int flag [64];
int main(){
    for(int i=0;i<64;i++){
        flag [i] = 0;}
    pthread_init();
    pthread_spawn(fun,0);
    pthread_join();
    pthread_halt();
    for(int j=0;j<64;j++)
        if(flag [j] !=123 +j) printf("%d err\n",j);
    printf("All done\n");
    return 0;
}
```

[slave.c]

```
#include<slave.h>
extern int flag [64];
__thread int xx=123;
int fun(){
    xx = xx + _PEN;
    flag [_PEN] = xx;
}
```

4.5.3 加速线程库的示例：快速线程组

[host.c]

```
#include<pthread.h>
extern void SLAVE_FUN(fun());
int flag [64];
int main(){
    for(int i=0;i<64;i++){
        flag [i] = 0;}
    pthread_init();
    pthread_enter64();
    pthread_spawn64(fun);
```

```

    pthread_join64();
    pthread_leave64();
    pthread_halt();
    for(int j=0;j<64;j++)
        if(flag [j] !=123 +j) printf("%d err\n",j);
    printf("All done\n");
    return 0;
}

```

[slave.c]

```

#include<slave.h>
extern int flag [64];
__thread int xx=123;
int fun(){
    xx = xx + _PEN;
    flag [_PEN] = xx;
}

```

4.5.4 大共享模式程序示例 A： 1 主核+384 从核

[host.c]

```

#include<pthread.h>
extern void SLAVE_FUN(fun());
int flag[6][64];
int main(){
    for(int i=0;i<6;i++)
        for(int j=0;j<64;j++)
            flag[i][j] = 0;
    pthread_init_all();
    pthread_spawn_all(0x1,fun,0);
    pthread_spawn_all(0x2,fun,0);
    pthread_spawn_all(0x4,fun,0);
    pthread_spawn_all(0x8,fun,0);
    pthread_spawn_all(0x10,fun,0);
    pthread_spawn_all(0x20,fun,0);
    pthread_join_all();
    for(int i=0;i<6;i++)
        for(int j=0;j<64;j++)
            if(flag[i][j] !=123+i+j) printf("%d: %d err\n",i,j);
    printf("All done\n");
}

```

```

        return 0;
    }

[slave.c]
#include<slave.h>
extern int flag[6][64];
__thread int xx=123;
int fun(){
    xx = xx + _CGN+_PEN;
    flag[_CGN][_PEN] = xx;
}

```

附加说明：运行时需传递加载器参数“-a 1”。

4.5.5 大共享模式程序示例 B：6 主核+384 从核

```

[host.c]
#include<athread.h>
#include<pthread.h>
extern void SLAVE_FUN(fun());
int flag[6][64];
void* mythread(int arg){
    int cgid = (int)arg;
    /*绑定该线程到指定核组*/
    SET_TO_SWCG(cgid);
    printf("I am CG%d\n",current_array_id());
    athread_spawn_all((0x1<<cgid),fun,0);
    athread_join_cgmask(0x1<<cgid);
}
int main(){
    for(int i=0;i<6;i++)
        for(int j=0;j<64;j++)
            flag[i][j] = 0;
    athread_init_all();
    pthread_t pt[6];
    int ret;
    for(int i=0;i<6;i++){
        ret = pthread_create(&pt[i],NULL,mythread,i);
        if(ret!=-1) printf("pthread_create %d err\n ",i);
    }
    for(int i=0;i<6;i++){

```

```

        printf("pthread_join %d \n ",i);
        pthread_join(pt[i],NULL);
    }
    for(int i=0;i<6;i++)
        for(int j=0;j<64;j++)
            if(flag[i][j] !=123+i+j) printf("%d: %d err\n",i,j);
    printf("All done\n");
    return 0;
}

```

[slave.c]

```

#include<slave.h>
extern int flag[6][64];
__thread int xx=123;
int fun(){
    xx = xx + _CGN+_PEN;
    flag[_CGN][_PEN] = xx;
}

```

附加说明：运行时需传递加载器参数“-a 1”。

第五章 SIMD 向量编程简介

5.1 概述

SIMD (Single Instruction Multiple Data) 是单指令流多数据流的缩写。SW39000 众核处理器主核支持 256 位 SIMD 向量扩展指令,从核支持 512 位 SIMD 向量扩展指令。该功能的增加,不仅能够降低功耗,而且在一定程度上提升了指令级并行的能力。本章给出简单的 SIMD 向量编程介绍,详细的介绍请参见《基础编译器 SIMD 用户编程手册》。

对于标量与向量的区别,如下图所示,标量的运算粒度为单个元素,向量的运算粒度为一组有序的标量元素。一个向量中各个元素是互不相关的,对当前向量元素的操作结果不影响到其他向量元素。比如有一个向量,我们要使向量中的每一个元素 A_i 都乘以一个标量 b ,那么 b 与 A_1 相乘的结果不影响 A_2 的结果,各自独立。



一条 SIMD 指令相当于一个小的循环,所以可以减少指令数,从而可以降低对指令访问带宽的要求。并且减少了由循环引起的控制相关。例如对于主核:

```
int a (N),b (N),c (N)
for(i=0; i < N; i++) {
    a[i]=b[i]+c[i]
}
```

上面的程序如果采用 SIMD 技术,则最终的实现相当于:

```
for(i=0; i<N; i=i+8){
    a[i+7:i]=b[i+7:i]+c[i+7:i]
}
```

可以发现,循环迭代次数减少为原来的八分之一,该循环的总指令数减少到原来的八分之一左右。

使用基础编译系统的 SIMD 向量编程功能时,需要 `#include <simd.h>` 头文件,并在编译时加上 `-msimd` 选项。

5.2 串程序向量化示例

一般情况下,串程序向量化的着力点是程序中的循环。

```
int vertex[n], sum;
for (i = 0; i < n; i++){
    sum=sum+vertex[i];
};
```

对于上面的循环体,非常简洁规整,可以进行向量化。向量化一个循环一般可以分以下步骤:循环展开和分裂、变量替换、操作替换。

5.2.1 循环展开与分裂

对上面的循环根据向量宽度进行循环展开与分裂，以在主核上的实现为例，变化为如下的形式：

```
int sum_temp[8], vertex[n], sum;
for (i = 0; i < n/8; i += 8){
    for(j = 0; j < 8; j++){
        sum_temp[j] = sum_temp[j] + vertex[8*i+j];
    }
};
for(j = 0; j < 8; j++){
    sum = sum + sum_temp[j];
}
for (i = (n/8)*8; i < n; i++){
    sum=sum+vertex[i];
};
```

该例子中如果 n 不能被 8 整除，将最后不够一次向量操作的部分分裂出来，前面的部分就可以向量化了。

5.2.2 变量替换

声明扩展向量类型变量有两种策略，一种是在算法一级把关键变量都替换成扩展向量类型变量，这种方法改造程序彻底，但对程序改动较大；一种方法是在需要向量化的地方才引入扩展向量类型变量用于 SIMD 操作，该方法牵涉的地方较小，有利于快速向量化和调试。

```
int sum_temp[8]; → intv8 vsum;
int vertex[n]; → intv8 tx;
intv8 为主核的定点整形向量。
```

5.2.3 操作替换

若不是全局替换，在用 SIMD 操作替换普通操作之前，需要把标准类型变量映射（或复制）到扩展向量类型变量上，操作完之后，再把扩展类型向量变量映射（或复制）回标准类型变量。如：

```
simd_load (tx, &(vertex[8*i])); //将标准类型映射到扩展向量类型
simd_store (tx, &(vertex[8*i])); //将扩展向量类型映射到标准类型
通过上述步骤，对于示例的标量循环，最终的向量化程序如下：
int vertex[n], sum;
//扩展向量数据类型声明
intv8 vsum, tx;
//对能被 8 整除的部分进行向量化
```

```

for (i = 0; i < n/8; i += 8){
    //将 vertex[8*i]~ vertex[8*i+7]的连续 8 个标量从内存映射加载到向量中
    simd_load (tx, &(vertex[8*i]));
    //向量加法
    vsum = vsum + tx;
};
//调用扩展向量函数接口对向量中的 8 个标量进行规约求和
sum = simd_reduc_plusw(vsum);
//对于不能被 8 整除的部分仍然采用标量进行运算
for (i = (n/8)*8; i < n; i++){
    sum=sum+vertex[i];
};

```

5.3 对 C 语言的扩展

SW39000 众核处理器主从核分别提供了一百余条指令来实现 SIMD 的功能。为了用户更好的使用 SIMD，我们在编译器中提供了一整套方案，通过直接定义数据类型、调用内部函数，或者通过编译器自动的向量识别，来满足用户的需求。

5.3.1 数据类型扩展

编译器所支持的向量数据类型及相关说明可参加 2.6 节内容。

5.3.2 对界

使用 SIMD 进行向量编程时，向量的访存操作对于内存地址有一定的对界要求，用户需要知道这些限制，否则程序容易出错。因为编译器不会在对界方面产生报警或错误信息，向量类型的数据在使用过程中是否出现了不对界问题需要用户判断。

对于主核而言，向量类型的数据项在内存中除了 floatv4 类型是 16 字节对界，其余都是 32 字节对界。编译器保证用向量类型定义的变量在内存中是 32 字节（16 字节）对界的。包含向量类型的结构体（struct）/联合（union），编译器保证它们是 32 字节（或 16 字节）对界的（必要的时候在内部进行了垫塞）。对于从核而言，向量类型的数据项在内存中除了 floatv8 类型是 32 字节对界，其余都是 64 字节对界。编译器保证用从核向量类型定义的变量在内存中是 64 字节（32 字节）对界的。包含向量类型的结构体（struct）/联合（union），编译器保证它们是 64 字节（或 32 字节）对界的（必要的时候在内部进行了垫塞）。

在主核架构中，不对界的向量 Load/Store 会引发异常，操作系统收到异常信号后就会将这些 Load/Store 拆分成标准类型的 Load/Store，从而大大降低性能。在从核架构中，不对界的向量 Load/Store 会引发不对界访存异常，程序会直接报错退出。因此在向量编程时，编译程序可以通过添加 -faddress_align=n 选项，使得所有类型数组、结构体、联合体等的首地址是 n 字节对界的。

5.3.3 对 c 操作与内部函数的扩展

对于 c 语言中针对标量数据类型的操作，绝大部分都可以直接应用于向量数据类型上，例如取大小的 `sizeof()`、赋值操作、取地址符号 `&x`，指针等。此外对于常用的运算符操作，也支持从标量到向量数据类型的扩展，例如基本的算术操作（+、-、*、/）、逻辑操作（&、|、^等）、移位操作（>>、<<）、比较操作（==、<=、<等）等，当然，针对主从核不同的处理器结构所支持的向量指令的不同，部分运算符并不是对所有的向量数据类型都支持，具体情况请参见《基础编译器 SIMD 用户编程手册》。

对于扩展数据类型上的每一种运算操作我们都扩充了相应的内部函数或宏定义接口，供编程时调用。编译器成功安装后，只需要将“`simd.h`” `include` 到程序中就可以使用这些扩展数据类型和扩充的内部函数或宏定义了。例如上面示例程序中的 `vsum = vsum + tx` 也可以通过 `vsum = simd_vaddw(vsum, tx)` 来实现，其效果是一样的。但是对于 `sum = simd_reduc_plusw(vsum)` 这样的向量规约求和操作，就没有办法通过通用的运算操作符来实现。运算符之外扩充的内部函数主要用于以下功能的实现上：

- 标准类型数据与扩展向量类型数据的映射存取，主要是针对硬件提供的向量访存指令；
- 向量数据类型的初始赋值操作；
- 向量数据类型的条件选择操作，整理操作等；
- 向量数据类型的打印功能。

5.3.4 数据类型转换

C 语言中支持标准数据类型之间的互相转换，扩展到向量数据类型之后，编译器同样支持向量数据类型之间以及向量数据类型与标量数据类型之间的转换。需要注意的是，与扩展向量数据类型相关的数据类型转换，每一种转换都有不同的语义，用户需要加以区分。具体的不同的转换的含义请参见《基础编译器 SIMD 用户编程手册》。

5.4 程序向量化限制

5.2 节给出了一个循环向量化的示例，但是并非所有的循环都可以向量化。循环的向量化会受到很多方面的限制。

1、硬件限制

向量化受限于硬件的限制。在 SIMD 扩展中，向量化访存操作必须是地址连续的，且要求一定的对界要求。这意味着虽然有些循环是可向量化的，但具体针对国产众核体系结构，需要进行程序变换后才能向量化。另外，由于定点向量类型的指令集是不完备的，例如不支持定点向量的乘法、除法等，这样的循环就没有办法向量化。

2、程序相关性限制

向量化的基础是相关性分析，循环要能够向量化，要求将要被向量化的几次循环迭代之间不能存在相关性。下面这样的循环是无法进行向量化的。

```
for (i=1; i<N-1; i++){
```

```
data[i] = data[i-1]*0.25 + data[i]*0.5 + data[i+1]*0.25;  
}
```

3、循环结构限制

循环可以是常用的 `for`, `while-do` 或 `repeat-until` 结构, 也可以是使用 `goto` 和标号组成的。然而, 循环必须是只有一个入口和一个出口时才能被向量化。

4、循环退出条件限制

循环退出条件决定循环执行的迭代次数。向量化需要迭代次数是可以计算的, 也就是说迭代次数必须是下述表达式:

- a) 常数;
- b) 循环不变量;
- c) 外层循环控制变量的线性函数。

第六章 加载器

6.1 概述

因为从核上未运行标准的操作系统，也无法通过 `execve` 方式来加载运行一个程序，运行的程序需要通过一个独立的加载器 `swrun` 来进行加载。

`swrun` 运行在主核系统区，通过它将用户可执行程序映像加载到核组内存，并进行从核资源、各类空间以及堆栈的申请和初始化，最后跳转到程序入口 PC 开始执行，加载器自动识别并支持单纯主核代码、主从混合代码、以及单纯从核代码，并支持对它们的加载和执行。

6.2 选项介绍

6.2.1 资源申请相关选项

- 1) -E 选项：指定申请从核的数量，且从最低可分配的从核号（一般是 0 号 PE）开始顺序分配；
- 2) -F 选项：用于指定申请从核位图的形状。

6.2.2 主存空间相关选项

核组内存根据空间属性一共分成四种类型：

- 1) 交叉段：通过 -T 指定
- 2) 核组共享段：通过 -s 指定
- 3) 核组私有段：通过 -t 指定
- 4) 只读段：通过 -r 指定

这四种空间指定的数值大小单位都是兆（M）；如果不指定的话，对核组共享段和只读段默认都是 64M；私有段大小表示是每个启动的从核分配的大小，不指定的默认值为 2M；交叉段是全片 6 个 CG 共享的，对同一道作业，要求每个 CG 指定的交叉段大小相同，不指定的默认值是 0M。

主核 `malloc` 默认分配的空间在核组共享段上，为了满足不同应用的需求，加载器提供了选项指定 `malloc` 分配的空间在交叉段上，具体如下：

-o (--malloccross) :

- o 1: 提供给 `linpack` 使用，`malloc` 分配空间使用本核组交叉段空间。
- o 2: 主核 `malloc` 分配的空间使用交叉段空间。

6.2.3 LDM 和 Cache 配置相关选项

-L (--ldm_share_mode) 连续共享 LDM 模式（0、1、2、3、4、5）

- 0: 单簇模式
- 1: 双簇模式
- 2: 四簇模式

4: 全阵列簇模式

5: 全阵列行模式

-l (--ldm_share_size): 指定连续共享 LDM 大小 (0、4、8、16、32、64、128) KB。

-c (--cache_size): 从核数据 Cache 容量大小配置, 支持 0KB、32KB 和 128KB 三档配置, 若不设置该选项, 加载器默认配置从核数据 Cache 容量为 0KB。

6.2.4 栈空间选项

主核栈空间和从核栈空间分别使用 -K 和 -S 来指定, 用户需要保证自己的程序分配到足够的主从核栈空间, 否则程序可能出现不可预期的错误。如果不指定, 主核栈空间默认为 16M, 从核栈空间默认为每个核 64K。

-b 选项用于将从核程序栈指针指向 LDM 空间, 这样可以使对栈变量的访问转为对 LDM 空间上的访问, 提升程序性能。需要注意的是, 如果栈空间太大, 程序运行过程中可能会对原有用户的 LDM 空间的使用造成影响, 产生不可预知的错误。尤其是动态调整从核连续共享 LDM 模式/容量以及从核 cache 容量时, 会额外动态申请与栈空间大小相同的 LDM 空间进行栈空间的转移工作。对此, 编译器提供了栈深度分析工具和栈溢出检查功能辅助用户使用。

6.2.5 从核浮点状态控制

-m (--mieee) 选项提供从核自陷模式的控制, 指定 -m 2 时, 将浮点控制状态寄存器 fcpr 的最后两位设为 01, 允许除不精确结果之外的所有浮点算术异常自陷, 相当于编译器使用 -mieee 选项; 指定 -m 1 时, 将 fcpr 最后两位设为 00, 允许所有浮点算术异常自陷, 相当于编译器使用 -mieee-with-inexact 选项; 其他所有值将不对默认的 fcpr 进行修改。

6.2.6 swprof 性能调优相关的选项

-P (--prof) 该选项用于对程序进行热点函数分析, 实现类似 -pg 选项的功能, 但用户程序却不需要重新加选项编译。-P 选项支持三个参数: “all”、“master”、“slave”, 将指示加载器进入不同的程序剖析模式, 获取对应的热点函数。

all: 对所有主从代码进行剖析, 获取主从热点函数

master: 只对主核代码进行剖析, 获取主核热点函数

slave: 只对从核代码进行剖析, 获取从核热点函数

-p (--pg) 该选项用于对程序进行热点函数分析, -p 等价于 -P all, 对所有主从代码进行剖析。

使用 -P 和 -p 选项后, 将在用户程序运行结束后在屏幕输出热点函数统计信息, 如下图所示:

```

===== (Node 0:CG 0) Hot Function List=====
No. InstSize(B) Time(s) State FunName
1 3216 20.99 ORIGIN slave_sha1_transform16
2 2180 3.90 ORIGIN slave_main_kernel
3 1020 2.74 ORIGIN slave_hmacsha1_3
4 764 1.94 ORIGIN slave__AES_set_decrypt_key_128
5 396 1.69 ORIGIN slave_PSK
6 108 1.53 ORIGIN slave_interrupt_host
7 284 1.32 ORIGIN slave__AES_para_decrypt_128
8 1308 0.70 ORIGIN slave_hmacsha1_1
9 892 0.65 ORIGIN slave_hmacsha1_2
10 428 0.58 ORIGIN athread_join
11 368 0.43 ORIGIN slave_AESDECL
12 1132 0.41 ORIGIN slave__psfs_parse_spec
13 476 0.33 ORIGIN slave__AES_set_encrypt_key_128
14 284 0.23 ORIGIN slave__AES_para_encrypt_128
15 128 0.04 ORIGIN slave__scan_ungetc
16 160 0.04 ORIGIN slave__divlu
17 820 0.03 ORIGIN slave_stdlib_strto_l
18 2180 0.03 ORIGIN slave__GI_vfscanf
19 244 0.02 ORIGIN slave_getenv
20 140 0.02 ORIGIN slave__remu
21 388 0.02 ORIGIN slave_Mess12
22 168 0.01 ORIGIN slave__store_inttype
23 268 0.01 ORIGIN slave__GI_ungetc
24 132 0.01 ORIGIN slave__init_scan_cookie
25 272 0.01 ORIGIN slave__GI_memset
26 664 0.01 ORIGIN slave_TiQuShuJu
27 84 0.01 ORIGIN slave_SHA1_init_begin
===== (Node 0:CG 0) End of HF List=====

```

-n (--hf_number) 指定统计的热点函数数目，将只统计前 n+1 个热点函数。

-N (--profnode) 指定参与统计热点函数的节点列表（逻辑节点号从小到大），例如'0,2,4-5'。

-f(--hf_reorder) 针对从核热点函数，进行热点函数代码段重排优化以缓解指令 cache 冲突，该优化需要和-p 或-P 选项同时使用才有效果。使用该选项后，将在热点函数统计信息之后打印重排的函数信息，如下所示：

```

22 168 0.01 ORIGIN slave__store_inttype
23 820 0.01 ORIGIN slave_stdlib_strto_l
24 296 0.01 ORIGIN slave_getc
25 132 0.01 ORIGIN slave__init_scan_cookie
26 244 0.01 ORIGIN slave_getenv
27 272 0.01 ORIGIN slave__GI_memset
28 84 0.01 ORIGIN slave_SHA1_init_begin
===== (Node 0:CG 0) End of HF List=====

Reorder function slave_sha1_transform16, move from 0x4ffff0415640 to 0x4ffff03fb850
Reorder function slave_main_kernel, move from 0x4ffff0412570 to 0x4ffff03fc4e0
Reorder function slave_hmacsha1_3, move from 0x4ffff0410d40 to 0x4ffff03fcd70
Reorder function slave__AES_set_decrypt_key_128, move from 0x4ffff0415168 to 0x4ffff03fd170
Reorder function slave_PSK, move from 0x4ffff0411140 to 0x4ffff03fd470
Reorder function slave_interrupt_host, move from 0x4ffff04162d0 to 0x4ffff03fd600
Reorder function slave__AES_para_decrypt_128, move from 0x4ffff0414f30 to 0x4ffff03fd670
Reorder function slave_hmacsha1_1, move from 0x4ffff04104a0 to 0x4ffff03fd790
Reorder function slave_hmacsha1_2, move from 0x4ffff04109c0 to 0x4ffff03fdbc0
Reorder function slave_AESDECL, move from 0x4ffff0410290 to 0x4ffff03fe030
Reorder function slave__AES_set_encrypt_key_128, move from 0x4ffff0415464 to 0x4ffff03fe1a0
Reorder function slave__psfs_parse_spec, move from 0x4ffff0419a34 to 0x4ffff03fe380
Reorder function slave__AES_para_encrypt_128, move from 0x4ffff041504c to 0x4ffff03fe7f0
Reorder function slave__scan_ungetc, move from 0x4ffff04199b4 to 0x4ffff03fe910
Reorder function slave__GI_vfscanf, move from 0x4ffff0418f40 to 0x4ffff03fe990
Reorder function slave__GI_strlen, move from 0x4ffff0417a00 to 0x4ffff03ff220
Reorder function slave_Mess12, move from 0x4ffff04112d0 to 0x4ffff03ff280
Reorder function slave__divlu, move from 0x4ffff0416c38 to 0x4ffff03ff410
Reorder function slave_TiQuShuJu, move from 0x4ffff0411720 to 0x4ffff03ff4b0
Reorder function slave__remu, move from 0x4ffff0416d30 to 0x4ffff03ff750
Reorder function slave__store_inttype, move from 0x4ffff041b9cc to 0x4ffff03ff7e0
Reorder function slave_stdlib_strto_l, move from 0x4ffff041ac90 to 0x4ffff03ff890
Reorder function slave_getc, move from 0x4ffff041a8d4 to 0x4ffff03ffbd0
Reorder function slave__init_scan_cookie, move from 0x4ffff04197c4 to 0x4ffff03ffd00
Reorder function slave_getenv, move from 0x4ffff0417a58 to 0x4ffff03ffd90
Reorder function slave__GI_memset, move from 0x4ffff0417880 to 0x4ffff03ffe90
Reorder function slave_SHA1_init_begin, move from 0x4ffff0410400 to 0x4ffff03fffa0

```

加载器会将重排后的代码段重新写回原目标码，用户重新提交运行程序（无需再指定热点函数相关选项），可以得到优化后的执行情况。

-D (--ipad) 在从核热点函数重排优化过程中，进行指令垫塞优化以进一步缓解指令 cache 冲突，该优化需要和-p/-P 以及-f 选项一起使用才有效果。该选项需指定进行指令垫塞拷贝的份数，取值范围 0~8，超出范围将使用默认值 0。使用该选项后，与-f 选项类似，将在热点函数统计信息之后打印多次拷贝重排的热点函数信息。

-U (--user_hf_reorder) 该选项执行用户自定义热点函数重排优化，不通过热点函数统计，

用户自行指定需要重排的从核热点函数名。需要提供函数名列表参数，例如 'slave_k1,slave_k2'。该选项将在用户程序运行之前执行热点函数重排，重写目标码，并打印出相关重排信息，用户无需重新运行目标码，本次运行就将得到优化后的执行情况。

-Q (--swprof_quiet) 该选项打开后将不在屏幕输出关于 swprof 的打印信息，而是将其输出到以 swprof_nodeid_cgid.log 方式命名的文件中。

swprof 性能调优相关选项说明：不建议将用户指定的热点函数重排优化-U 和自动的重排或堵塞优化-f/-D 一块使用，因为互相影响可能得到无法预知的结果。所有的相关优化只会执行一次，即优化过的目标码再次执行 swprof 相关优化将不起作用。如需再次尝试，请重新编译生成目标码。

6.2.7 大共享模式的运行选项

-a 1 指定大共享运行时模式。

该模式下，一个处理器上只运行一个进程，该进程运行在某一个 CG 上（一般是 0CG），但该进程可以通过大共享模式的线程库接口启动全片 6CG 的从核资源（详见 4.1.17-4.1.21），该模式下的默认空间依然是核组共享空间，如果想使用全片交叉共享空间，可通过 __cross 定义交叉段静态变量，或者加载器选项“-o 2”指定动态申请空间为全片交叉空间。

6.2.8 加载器配置相关选项

-I (--swrun_info) 显示进入加载器开始运行的打印信息，供调试用

-C (--config) 指定加载器选项的配置文件名字（不需要在命令行给出过多选项，方便用户使用），例如-C ./config-file。

6.2.9 加载器特别说明

基础编译系统支持在一个核组上运行多个加载器，每个加载器可加载不同的单从核程序或者主从混合程序。

当加载器申请的空间总大小超出核组空间大小时，会报错申请空间失败，后续的加载器无法继续加载程序运行。

当新的加载器申请从核资源与已有加载器冲突时，也会因申请资源失败而报错退出。

第七章 众核编译系统工具

7.1 混合程序反汇编器

基础编译系统面向的异构众核架构主从核指令集不兼容，对于主从核混合程序，单纯使用主核反汇编器或者从核反汇编器都不能正确反汇编混合程序，因此编译系统专门提供了针对混合程序的反汇编器 `swobjdump`。

`swobjdump` 能够区分混合程序中的主核代码与从核代码，并分别调用主核反汇编器与从核反汇编器进行反汇编处理，从而得到混合程序正确的反汇编代码。

`swobjdump` 只支持 `-D` 和 `-d` 选项，一次只能处理一个混合程序目标码，其中 `-D` 反汇编所有段，`-d` 只反汇编代码段。用法：

`swobjdump <-D/-d> <file>`，或者：

`swobjdump <file> <-D/-d>`

7.2 ldmreport

由于 LDM 空间不允许超过 256k，而 `ldmreport` 可以检测 LDM 空间使用多少，并在 `ldm` 空间溢出 256k 时报错，超过 128k 时报警。

具体使用如下所示：

这是一个 `ldm` 用例 `c.c`，里面定义了六个静态变量：

```
#include "slave.h"

#define LOOP 100

__thread_local volatile int local_rply;
__thread_local volatile int remote_rply;
//__thread_local long long a[16384 + 32];
__thread_local long long a[32];

__thread_local unsigned long long reg_tmp[2];
__thread_local unsigned long long la[LOOP];
__thread_local unsigned long Rb[LOOP], Rc[LOOP];
```

首先对其进行编译，可执行 `swgcc -mslave -msimd -faddress_align=64 -c c.c`，将会得到 `c.o`

然后执行 `ldmreport c.o`，将会输出如下结果：

```
LDM local variable list is:
indx:0      addr:0x40      size:4      name:local_rply
indx:1      addr:0x44      size:4      name:remote_rply
indx:2      addr:0x80      size:256     name:a
indx:3      addr:0x180     size:16     name:reg_tmp
indx:4      addr:0x1c0     size:800    name:la
indx:5      addr:0x500     size:800    name:Rb
indx:6      addr:0x840     size:800    name:Rc
==== LDM local max align: 64B, data num: 7, total size: 2944B
```

即输出每个变量所用 `ldm` 空间大小。如果总的 `ldm` 空间占用很大（如超过 128k），将会报警，如下图所示：

```
LDM local variable list is:
indx:0      addr:0x40      size:4      name:local_rply
indx:1      addr:0x44      size:4      name:remote_rply
indx:2      addr:0x80      size:131328  name:a
indx:3      addr:0x20180   size:16     name:reg_tmp
warning!!!the ldsym.addr overflow 128k!!!
indx:4      addr:0x201c0   size:800    name:la
warning!!!the ldsym.addr overflow 128k!!!
indx:5      addr:0x20500   size:800    name:Rb
warning!!!the ldsym.addr overflow 128k!!!
indx:6      addr:0x20840   size:800    name:Rc
warning!!!the ldsym.addr overflow 128k!!!
==== LDM local max align: 64B, data num: 7, total size: 134016B
```

如果总的 ldm 空间占用过大，超过了 256k，将会有以下提示：

```
=====LDM INFO begin=====
LDM local variable list is:

indx:0      addr:0x40      size:262240  name:a

==== LDM local max align: 8B, data num: 1, total size: 262304B

==== ldm_malloc start addr: 0x400c0

==== ldm_share start addr: 0x300000000000 align: 0B size: 0B

=====ALL LDM SIZE: 262336B(private),0B(shared)]=====
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
Your program's static ldm data overflow 256K, it cannot be loaded TO SW9
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
```

7.3 checklen

从核 L1 指令 Cache 典型配置为 32KB，当被调用函数的指令段大小超过该容量时，会发生指令 Cache 替换而导致性能下降。因此编译系统提供了查看目标码中某从核函数及所有其调用函数指令段总大小的 checklen 工具。

checklen 工具用法如下：

```
checklen <a.out> <func_name>
```

checklen 能够自动识别<a.out>中的所有从核函数，从中递归地找出<func_name>函数及所有其调用函数，计算所有调用函数的起始地址及指令段大小，并在最后计算出所有调用函数指令段的总大小。用户可以根据上述输出判断<func_name>函数执行时是否发生指令 Cache 替换，从而指导对程序性能进行调优。

checklen 工具使用实例如下：

```
[gfscomp@sn01 zhouwh]$ checklen a.out fun
```

calldepth	func_name	start_addr	length
0	fun	00004ffff0410160	68
1	fun1	00004ffff04101a4	68
2	fun2	00004ffff04101e8	88

```
The ICache occupation is: 224 B
```

7.4 从核栈深度分析功能

从核栈深度分析功能可以显示出用户程序从核函数的调用关系图，以及各个函数的最大

栈深度，为用户优化程序提供参考。

支持单从核程序及主从混合程序。

使用方式（链接时使用选项）：

-Wl,--stack-analysis

将会按照从大到小顺序打印出所有函数调用关系图中根节点的栈深度以及整个程序的最大栈深度需求。

```
Stack analysis will ignore the call from slave_fun1 to slave_fun1
=====slave-core func stack depth analysis=====
stack depth for call graph root nodes:
  slave_main: 401536
  slave__do_one_spec: 1024
  slave__cs_seek: 512
  slave__start: 464
  slave__fp_out_narrow: 384
  slave__charpad: 384
  slave__cs_read: 320
  slave__cs_write: 320
  slave__cs_close: 320
  slave__syscall_exit: 192
  slave__promoted_size: 128
  slave__is_equal_or_bigger_arg: 128
  slave__divlu: 48
  slave__divl: 48
  slave__divwu: 48
  slave__divw: 48
  slave__remwu: 32
  slave__remw: 32
Maximum stack required is 401536
=====
```

如果想查看函数调用关系及其他函数的详细栈深度信息，可以使用如下选项：

-Wl,-Map -Wl,mymapfile

在 mymapfile 中可以看到如上信息：

```
      slave__stdio_transzw
slave__GI_printf: 320 1344
calls:
  * slave__GI_vfprintf
slave_test: 400128 401472
calls:
  * slave__GI_printf
slave_fun3: 64 64
slave_fun1: 64 64
slave_fun2: 64 128
calls:
  * slave_fun1
slave_main: 64 401536
calls:
  * slave_test
    slave_fun3
    slave_fun2
    slave_fun1
```

无法识别的栈深度情况包括：

- 1) 函数指针；
- 2) 嵌套函数调用将被忽略；
- 3) 其他只有在运行时才能得知的动态栈深度调整。

7.5 从核栈在 LDM 上的溢出检查

当从核栈空间置于 LDM 上时，也即是程序运行时使用了加载器选项“-b 1”时，提供检查机制，为用户报告哪个函数的栈空间溢出到了 ld_malloc 的动态空间或 LDM 的静态

空间。

使用方式：

编译时使用选项-fcheck-ldm-stack，只针对从核程序

单从核课题链接时增加-lslave

运行时加载器使用-b 1 选项

程序运行时首先检查从核函数的栈空间是否溢出到 ldm_malloc 的动态空间，没有则继续检查从核函数的栈空间是否溢出到 LDM 的静态空间。

如果当程序调用函数 fun1 时发现 fun1 的栈空间已经溢出到 ldm_malloc 的动态空间内了，则会在 fun1 函数执行前就跳转到异常报错函数，打印如下：

```
Ldm_malloc space will be overflowed by stack on ldm [function pc: 4ffff0410488]
```

```
See function name : swaddr2line -f -e a.out [function pc]
```

以上打印说明程序在 pc=4ffff0410488 的地方开辟栈空间，栈空间溢出到了 ldm_malloc 动态空间内，使用命令 swaddr2line -f -e a.out 0x4ffff0410488 可以查到该 pc 对应的函数为 slave_fun1。

如果当程序调用函数 fun1 时发现 fun1 的栈空间已经溢出到 LDM 静态空间内了，则会在 fun1 函数执行前就跳转到异常报错函数，打印如下：

```
Stack on ldm will be overflowed [function pc: 4ffff0012ec4]
```

```
See function name : swaddr2line -f -e a.out [function pc]
```

以上打印说明程序在 pc=4ffff0012ec4 的地方开辟栈空间，栈空间溢出到了 LDM 静态空间内，使用命令 swaddr2line -f -e a.out 0x4ffff0012ec4 可以查到该 pc 对应的函数为 slave_fun1。

7.6 从核汇编器特殊选项

-Wa,-slave_disp_noalign

从核访存指令对偏移有对界要求，如 ldl 的偏移地址低 3 位[2:0]默认为 0，之前的处理是从核汇编器遇到偏移不满足对界要求的就报错退出，如“ldl \$1,4(\$2)”，汇编器会报“Fatal error: disp[2:0] of ‘ldl’ is not 0 ”错。

实际应用中会出现下面这种情况，如对于“ldl \$1,4(\$2)”，其中(\$2+4)地址是满足 8 字节对界要求的，但是汇编器还是会报错。针对这种情况，即访存指令偏移不对界，但是用户保证基址+偏移满足对界要求的情况下，使用-Wa,-slave_disp_noalign 选项，汇编器会处理成 ldi+访存两条指令。

如“ldl \$1,4(\$2)”，使用-Wa,-slave_disp_noalign 选项，汇编器处理成“ldi \$63, 4(\$2); ldi \$1, 0(\$63)”，其中\$63 为编译器保留的内部寄存器。

该选项默认打开。

7.7 基于控制流图的代码静态特征提取

基础编译系统支持代码静态特征提取。通过对代码控制流图的分析，以函数为单位提取

代码中的静态特征。

编译选项：

-fextract-feature

示例：

\$ swgcc -fextract-feature example.c

代码静态特征的具体意义请参见《基础编译器常用编译选项手册》。

7.8 version-spy 版本检测

编译器版本更新较快，版本更新提供新功能或解决用户问题的同时，也可能会引入其他方面的问题，给用户使用带来困扰。为此我们提供 version-spy 版本检测工具，用来检测目标代码里面的.o 与.a 文件是使用的哪个版本的编译器编译的，在用户遇到编译器版本更新相关的问题时，方便定位。

工具使用方法：

`version-spy <a.out>`

打印示例：

```
=====
Library info:
[0]:/usr/sw/swgcc/swgcc710-tools-SEA-1041/usr/bin/../lib/gcc/sw
_64sw6a-sunway-linux-gnu/7.1.0/libgcc.a
=>size:955130 time:Tue Dec 15 10:58:20 2020

[1]:/usr/sw/swgcc/swgcc710-tools-SEA-1041/usr/bin/../lib/gcc/sw
_64sw6a-sunway-linux-gnu/7.1.0/libgcc.a
=>size:955130 time:Tue Dec 15 10:58:20 2020

=====
=====

swgcc version:
==> [0]GCC: (GNU) 7.1.0 20170502 (swgcc-1041 Committer: qia
nh <qianh@example.com> by comp9 at localhost.localdomain on Mon Dec 14
21:09:07 CST 2020)
=====
```

第八章 常用优化支持

本章介绍编译系统常用的优化支持，包括循环展开、乘加融合等传统优化，也包括针对架构开发的编译优化。更为详细的编译选项和优化支持可参见《基础编译器常用编译选项手册》。

8.1 循环展开优化

国产众核基础编译器支持多种场景的循环开展优化，常用优化选项包括以下几种：

a) **-funroll-loops**

循环展开优化。

示例：

```
$ swgcc -funroll-loops example.c
```

b) **-funroll-all-loops**

激进的循环展开优化。

示例：

```
$ swgcc -funroll-all-loops example.c
```

c) **-funroll-nest-loops**

面向多重循环结构的循环展开优化。

示例：

```
$ swgcc -funroll-nest-loops example.c
```

d) **--param max-unroll-times=n**

设定最大的循环展开次数，默认为 8。

示例：

```
$ swgcc -funroll-loops --param max-unroll-times=16 example.c
```

8.2 乘加融合优化

国产众核基础编译器支持对乘加操作进行指令融合优化，利用硬件乘加单元，将乘法和加法两条指令合并为一条乘加指令。

编译选项：

-mfma

示例：

```
$ swgcc -mfma example.c
```

8.3 自动向量化

国产众核基础编译器支持主从核的自动向量化功能，使用方式如下：

编译时在-O2 选项优化的基础上加上-msimd -ftree-vectorize 选项或者直接使用-O3 -msimd 选项即可打开编译器的自动向量化功能。

编译选项：

-msimd -ftree-vectorize/ -O3 -msimd

示例：

```
1) $ swgcc -msimd -ftree-vectorize example.c
```

```
2) $ swgcc -O3 -msimd example.c
```

8.4 结构体重组优化

国产众核基础编译器支持结构体重组优化，将访问次数较多的热点结构体域剥离，组成新的结构体，增强数据访问的局部性。“访问次数较多”的阈值可配置，默认为 90%。

编译选项：

a) **-fipa-type-escape -fipa-struct-peeling**

b) **--param struct-reorg-cold-struct-ratio=<value>**

注：<value>取值介于 0-100，默认为 10

示例：

a) **\$ swgcc -fipa-type-escape -fipa-struct-peeling example.c**

b) **\$ swgcc -fipa-type-escape -fipa-struct-peeling --param struct-reorg-cold-struct-ratio=20 example.c**

8.5 从核 Cache 预取优化

国产众核基础编译器支持对连续数组访问进行预取，将主存中数据预取到从核 Cache 中。

编译选项：

-fprefetch-loop-arrays

示例：

\$ swgcc -mslave -fprefetch-loop-arrays example.c

附加说明：

➤ 预取优化默认打开循环展开优化，可以通过调整循环展开次数影响预取效果。

➤ 预取参数可以通过以下编译参数进行调整：

--param prefetch-latency-slave=n 设定预取延迟为 n 条指令，默认为 800。

该参数影响预取距离。

--param simultaneous-prefetches-slave=n 同时预取的指令数为 n，默认为 15。

--param l1-cache-slave-size=n 从核 L1cache 大小，单位为 KB，默认为 32。

8.6 从核 Cache 写暗示优化

国产众核基础编译器支持从核 Cache 的写暗示优化，对连续大块数据的 Cache 写操作进行优化，提高 Cache 访问性能。

编译选项：

-fcache-hint

示例：

\$ swgcc -mslave -fcache-hint slave1.c slave2.c

8.7 从核 Cache 属性动态配置

国产众核基础编译器支持从核变量的可 Cache 属性和不可 Cache 属性的动态配置，通过编译选项加编译指导语句配合使用。

编译选项：

-fdynamic-uncache-symbols

编译指导语句：区分数据默认是 cache 和 uncache 的两种情况，设置两组编译指导语句。

1、数据默认为 cached

#pragma uncache “xxx” ----将符号 xxx 访问变换为 uncache 访问

#pragma resume_cache “xxx” ----将符号 xxx 访问恢复为 cache 访问

2、数据默认为 uncached

#pragma cache “xxx” ---将符号 xxx 访问变换为 cache 访问

#pragma resume_uncache “xxx” ---将符号 xxx 访问恢复为 uncache 访问

3、编译指导语句参数说明

#pragma uncache “aa” 将符号 aa 的访问替换为 uncache 访问

#pragma uncache “aa@@foo” 将函数 foo 内符号 aa 的访问替换为 uncache 访问

示例：

编译命令

\$ swgcc -mslave -fdynamic-uncache-symbols slave.c

编译指示字

```
1 #include <stdio.h>
2 #include <slave.h>
3
4 int aa=3;
5 __uncached int dd=4;
6 __uncached int bb=5;
7 __uncached int cc[20];
8
9 int foo(){
10     #pragma cache lld@@foo"
11     flush_slave_cache();
12     int ret = aa+dd;
13     #pragma resume_uncache lld@@foo"
14     flush_slave_cache();
15     return ret;
16 }
17
18 int main()
19 {
20     printf("dd is %d\taa is 0x%lx\n",dd, &aa);
21     aa = dd+4;
22     #pragma uncache lla@@main"
23     flush_slave_cache();
24     printf("dd is %d\taa is 0x%lx\n",dd, &aa);
25     aa = bb+5;
26     cc[10] += foo();
27     #pragma resume_cache lla@@main"
28     #pragma cache bb"
29     flush_slave_cache();
30     printf("bb is %d\tbb addr is 0x%lx\n",bb, &bb);
31 }
```

```

32      #pragma cache &c"
33      flush_slave_cache();
34      for(int i=0; i<20; i++)
35          printf("%d\n",cc[i]);
36 }

```

附加说明：

- 该机制只面向对处理器体系结构较为熟悉的高级用户。
- 该机制只面向从核 Cache 结构。
- 该机制不保证数据访问的正确性，需要在属性变化前后添加 Cache 刷新语句或者 Cache 淘汰语句来保证 Cache 与主存之间的数据一致性。
- 该机制采用基于 pragma 实现的文本分析策略，一组编译指导语句确定了变换的空间，在此空间内的符号访问被转换为 uncache/uncached 访问，在此空间外，不做任何变换。
- ”#prama resume_xxx“为可选语句，如果不添加，则表示对应的”#prama xxx”作用范围将持续到文本结尾；若为”#prama xxx@@yyy“，则作用范围将持续到函数 yyy 文本结尾。（此处 xxx 表示 cache/uncache）。

8.8 从核代码链接时优化

国产众核基础编译器支持从核代码链接时优化，需要在编译从核文件及 hybrid 链接时使用。

编译选项：

-mslave-lto

示例：

\$ swgcc -mhost -c host.c

\$ swgcc -mslave **-mslave-lto** -c slave1.c slave2.c

\$ swgcc -mhybrid **-mslave-lto** host.o slave1.o slave2.o

8.9 从核代码分支预测优化

国产众核基础编译器支持从核代码分支预测优化，通过自动代码插桩，改变条件转移指令的预测方向，降低因分支预测失败导致的性能损失。

基于硬件支持，实现了两套分支预测优化机制：一是基于 hint 指令的分支预测优化，一是基于 pws 指令的分支预测优化。

编译选项：

1) **-fbranch-predict-hint**

2) **-mpws**

示例：

1) \$ swgcc -mslave **-fbranch-predict-hint** slave1.c slave2.c

2) \$ swgcc -mslave **-mpws** slave1.c slave2.c

8.10 FORTRAN module 访存优化

国产众核基础编译器支持将 Fortran 语言以 ldm__开头的 module 中的变量放在 LDM 中，

加速变量访问。

编译选项：

-fmodule-ldm

示例：

\$ swfortran -fmodule-ldm example.F90

8.11 功耗相关优化

国产众核基础编译器支持面向 SW39000 众核处理器特定结构的功耗优化，包括以下几种功耗优化选项：

a) -freg-bypass

面向寄存器旁路的指令调度优化。只支持从核代码，在-O2、-O3 情况下使能。

示例：

\$ swgcc -mslave -freg-bypass slave.c

b) -mslave-l0cache

基于 L0 指令 cache 的指令调度优化。只支持从核代码，在循环展开优化遍（例如-funroll-loops）使能的情况下有效。

示例：

\$ swgcc -mslave -mslave-l0cache slave.c

c) -flow-power

面向控制总线翻转的指令调度优化。只支持从核代码，在指令调度优化遍（例如-fschedule-insns、-fschedule-insns2）使能的情况下有效。

示例：

\$ swgcc -mslave -flow-power slave.c

8.12 快速数学库

国产众核基础编译器提供面向处理器体系结构特征优化的高性能快速数学库，使用方式如下：

链接时使用-lm（主核）、-lm_slave（从核）默认使用高性能数学库。

链接选项：

-lm（主核）、-lm_slave（从核）

示例：

1) \$ swgcc -mhost host1.o host2.o -lm

2) \$ swgcc -mslave slave1.o slave2.o -lm_slave

3) \$ swgcc -mhybrid host.o slave.o -lm -lm_slave

8.13 数学库函数细粒度多版本切换

国产众核基础编译器同时提供 GNU 标准数学库和面向处理器优化的高性能数学库，受限于冯诺依曼体系结构中浮点数据表示的精度误差，在特定条件下，二者的计算结果无法实现完全一致。通过编译选项配置，用户可以自定义配置代码中调用的部分数学函数的实现版本，辅助代码移植过程中的正确性验证。

该功能只支持从核代码。在-fmath-opt 不使能情况（默认情况）下，从核链接高性能数学库；在-fmath-opt 使能情况下，从核链接标准数学库，-fmath-opt=<str>指定的数学函数链

接高性能数学库中的实现。

编译选项：

-fmath-opt=<str>

示例：

```
$ swgcc -mslave -fmath-opt=sin,exp,cos slave1.c slave2.c -lm_slave  
(需要与工院数学库进行进一步联调)
```

8.14 支持动态链接程序

国产众核基础编译器支持代码的动态链接。主核默认采用动态链接模式，从核默认采用静态链接模式。使用-mhybrid_dynamic 可支持主从混合的动态链接模式。

备注：

使用动态库的程序通过 dlopen_hybrid 打开动态库。

示例：

1、生成动态库

```
swgcc -mhost -c host.c
```

```
swgcc -mslave -fPIC -shared -c slave.c
```

```
swgcc -mhybrid_dynamic host.o slave.o -shared -o test_hybrid.so
```

//示例程序 host.c

```
#include<athread.h>
```

```
int main() {}
```

```
int test_host(){
```

```
    printf("I am test_host!\n");
```

```
}
```

```
extern int a;
```

```
extern SLAVE_FUN(test_slave());
```

```
extern unsigned long addr;
```

```
int test_hybrid(){
```

```
    printf("I am test_hybrid.so in host!\n");
```

```
    athread_init();
```

```
    athread_spawn(test_slave,0);
```

```
    athread_join();
```

```
}
```

//示例程序 slave.c

```
#include<stdio.h>
```

```
#include<slave.h>
```

```
int test_slave(){
```

```
    printf("--- I am slave-core %d in test_hybrid.so\n",_PEN);
```

```
}
```

2、使用动态库

```
swgcc -c dl.c
```

```
swgcc -mhybrid_dynamic dl.o -ldl -o dl_test
```

//示例程序 dl.c

```
#include <stdio.h>
```

```

#include <stdlib.h>
#include <dlfcn.h>
void* dlopen_hybrid(char*,int);
int main(int argc, char **argv){
    void *handle;
    int (*test_hybrid)();

    handle = dlopen_hybrid("./test_hybrid.so", RTLD_NOW);
    dlerror();    /* Clear any existing error */
    printf("dlopen ./test_hybrid.so succeeded!\n\n");

    *(void **)(&test_hybrid) = dlsym(handle, "test_hybrid");
    printf("call test_hybrid function in test_hybrid.so!\n\n");
    (*test_hybrid)();
    dlclose(handle);

    exit(EXIT_SUCCESS);
}

```

需要注意的是：动态链接程序依赖的动态库路径可通过 `export LD_LIBRARY_PATH` 指定, 如果出现动态库版本不一致的问题, 需设置该路径指向最新版本的编译器的库路径。

修订记录

时间	版本号	修订内容	修改人
2020.03.22	0.1	基于上一代基础编译系统用户手册，完成基础版本	钱宏
2020.05.23	0.2	完成新版本用户手册章节架构和主体内容的设计调整	吴伟
2020.06.01	0.3	完成新版本用户手册各部分内容补充和完善	吴伟，钱宏，管茂林，朱琪，樊行健
2020.06.02	0.31	4.15 节修改创建线程组的附加说明，6.2.4 节添加栈空间选项在 cache 容量配置修改下的注意事项，8.12 节改为从核快速数学库未提供	管茂林
2020.06.03	0.32	7.5 节增加从核栈空间溢出到 ldm_malloc 动态空间的检查说明。	钱宏
2020.06.05	0.4	新增 4.1.20 获取核组号接口； 新增 4.2.8、4.2.9、4.2.10、4.2.11 的写暗示、私有段设置、从核阵列同步，从核阵列全同步等接口。	樊行健， 吴伟
2020.06.11	0.41	5.2 节修改循环分裂示例	管茂林
2020.06.12	0.42	2.6 节新增从核半精度浮点 BF16 格式支持。	钱宏
2020.06.16	0.43	8.14 节新增动态链接程序示例； 修改部分文字描述。	樊行健， 吴伟
2020.06.29	0.44	新增 4.1.13-4.1.16 带参数的快速线程组相关接口	杨涛
2020.06.30	0.45	4.3.4 节新增编程示例	钱宏
2020.07.01	0.46	新增 4.5.3 快速线程组示例	杨涛
2020.07.10	0.47	统一相关术语描述等	管茂林
2020.07.10	0.48	新增 4.2.12、4.2.13 节	钱宏
2020.09.15	0.49	从核数学库也默认使用工院快速数学库 更新 4.2.4、4.2.5 节相关注意事项	管茂林
2020.10.12	0.50	2.6 节新增从核半精度浮点除法和平方根运算支持。	钱宏
2020.10.14	0.51	新增 4.2.14 节从核半精度浮点 FP16 格式和 BF16 格式切换	钱宏
2020.10.16	0.52	修改 4.3.4 节 disp 参数说明	杨涛
2020.12.15	0.53	修改编译器名	杨涛